Coordination of Governed Agents Over Shared Context: Declarative Governance and Lyapunov-Tracked Finality

Jean-Baptiste Dézard*
Deal ex Machina SAS
jeanbapt@dealexmachina.com

March 2026

Abstract

The emergence of LLM-powered autonomous agents has exposed the limits of pipeline-based coordination: rigid DAGs cannot accommodate contradiction, evolving evidence, or formal convergence. We propose *declarative governance over shared immutable context*, a paradigm where autonomous agents reason over a common semantic graph, propose state transitions, and reach consensus through formal convergence mechanisms—without predefined execution sequences.

Seven architectural contributions enable this: (1) an append-only event log providing tamper-proof, fully replayable context; (2) a *bitemporal* CRDT-inspired semantic graph with dual valid-time and transaction-time axes, enabling point-in-time reconstruction for regulatory audit; (3) a *product order* $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ unifying governance levels (MASTER $\leq$ MITL $\leq$ YOLO) with a four-dimensional convergence rank into a single well-founded descent domain, with a deterministic *reduction kernel* that maps every proposal to Accept/Reject/Escalate through policy check, admissibility verification, and mode routing; (4) per-dimension vector finality that prevents compensation attacks (where one dimension over-improves to mask failures in another), implemented as a conjunction of per-dimension thresholds (τ_d, ϵ_d) with gate refactoring (GA^d, GC^d per-dimension; GB, GD, GE global), providing stronger guarantees than scalar disagreement-function tracking, with human-in-the-loop routing when convergence stalls; (5) Ed25519-signed finality certificates embedding policy-version hashes for cryptographic non-repudiation; (6) three-tier governance routing (deterministic rules, oversight agent, full LLM) with circuit-breaker fallback ensuring continuous operation without LLM availability; (7) a native Rust evaluation engine (compiled via napi-rs) executing all deterministic governance and convergence logic with zero LLM tokens, while TypeScript orchestrates I/O.

Unlike orchestration frameworks (AutoGen, CrewAI, LangGraph) that wire agents into chains or graphs, our system lets agents independently reason over shared state and propose actions evaluated by the reduction kernel against the product order. The construction draws on classical information-flow lattice models [6] and termination theory [1]: every governance-approved transition must descend in $\mathcal{M}$, guaranteeing termination of each convergence epoch. The kernel's determinism enables replay, audit, and formal verification independently of LLM stochasticity.

Validation on two scenarios—M&A due diligence (Project Horizon) and multi-period financial consolidation with bitemporal restatement—demonstrates that $V(t)$ reaches 0.0000 at resolution epochs (the first empirical confirmation of full intra-epoch convergence), exhibits a sawtooth trajectory across evidence epochs as predicted by the perpetual lifecycle model, and produces domain-independent dynamics confirmed across both scenarios. Five-gate prevention of false finality is ablation-confirmed, and governance modes produce distinct coverage–

*Source code: `https://github.com/DealExMachina/swarm-of-governed-agents`

autonomy positions. The architecture is validated by unit and integration tests and convergence benchmarks (see Appendix and experiments).

Crucially, finality is not terminal: new evidence, regulatory amendments, or periodic review triggers re-open convergence over the same graph, producing a chain of certified checkpoints that models the perpetual lifecycle of regulated knowledge. The resulting architecture is designed for regulated environments where temporal auditability, policy traceability, ongoing monitoring, and cryptographic proof of every decision point are operational requirements.

1 Introduction

The rapid maturation of LLM-powered agents—capable of reasoning, planning, and tool use—has created a coordination problem that traditional workflow engines were not designed to solve. Modern agents do not merely execute predefined tasks; they interpret context, generate hypotheses, and produce claims that may contradict each other. This capability demands a fundamentally different coordination model.

Current approaches fall into two categories, both inadequate:

Pipeline orchestration (Airflow, Temporal, Prefect) assumes static task graphs executed in topological order. When agents produce contradictory outputs, the pipeline either silently overwrites earlier results or fails entirely. There is no mechanism for tracking disagreement, no formal convergence guarantee, and no governance trail explaining *why* a particular conclusion was reached.

Agent frameworks (AutoGen, CrewAI, LangGraph) represent progress: they enable multi-agent conversations, role-based delegation, and tool use. However, they still wire agents into predefined topologies—sequential chains, hierarchical delegation, or cyclic graphs with hardcoded routing. Coordination logic is embedded in code, not governed by policy. There is no shared immutable context, no formal finality mechanism, and no fine-grained access control.

We propose a third path: *declarative governance over shared context*. Rather than sequencing agents, the system maintains:

1. **Shared bitemporal context:** An append-only event log and a CRDT-inspired semantic graph with dual temporality (valid time and transaction time), where confidence scores ratchet upward, contradictions are tracked explicitly, and no state is ever deleted—only superseded. Bitemporal queries enable point-in-time reconstruction for regulatory audit.

2. **Product order and reduction kernel:** A product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ unifies a governance-level chain $\mathcal{L}$ (MASTER $\leq$ MITL $\leq$ YOLO, ordered by permissiveness) with a four-dimensional convergence rank $\mathcal{A}$, forming a well-founded descent domain [4]. A deterministic *reduction kernel* evaluates every proposal against $\mathcal{M}$: policy rules check transition admissibility, the order verifies that the proposed state descends from the current state, and mode routing determines the verdict (Accept, Reject, or Escalate). This construction draws on classical lattice-based security models [2, 6] and rewriting-system termination theory [1]: every approved transition descends in $\mathcal{M}$, guaranteeing epoch termination. XACML combining algorithms [20] resolve multi-policy conflicts. Every decision produces an immutable, policy-version-stamped audit record.

3. **Formal convergence with five gates and perpetual lifecycle:** A disagreement function $V(t)$ tracks distance to targets across four weighted dimensions (proven a genuine Lyapunov function for the restricted CRDT system in Section 5.2). Monotonicity gates, evidence coverage, oscillation detection, quiescence, and minimum-content gates provide layered guarantees against premature finality. When autonomous convergence stalls, the system routes to

structured human review with full context. Finality is certified with Ed25519-signed JWS certificates—but finality is not terminal: new evidence, regulatory changes, or periodic review triggers re-open convergence over the same graph, producing a chain of certified checkpoints that models the perpetual lifecycle of regulated knowledge.

4. **Three-tier governance routing:** Deterministic rule evaluation (zero LLM tokens), lightweight oversight agent, and full LLM-backed governance agent, with circuit-breaker fallback ensuring continuous operation without LLM availability.

5. **Fine-grained access governance:** OpenFGA enforces who can read documents, propose transitions, approve decisions, and modify policies.

This paper presents the design, formal properties, and experimental validation of this system, with particular attention to its fitness for regulated environments.

At a deeper level, this design is not merely defensive engineering against failure cascades: at the level of state evolution, it instantiates the CALM criterion for coordination-free consistency. CALM establishes that a distributed program admits a coordination-free consistent implementation if and only if it is monotone [9, 15]. Our CRDT monotonicity invariants (Definition 2) implement precisely the monotone-write / threshold-read pattern that result identifies as necessary and sufficient: inflationary confidence updates are monotone writes; finality gates are threshold-style reads over shared state. The broader architecture (three-tier routing, Ed25519 certificates, OpenFGA access control) goes beyond what CALM requires; the minimality claim applies strictly to the state-evolution layer.

1.1 Contribution and Scope

The paper makes the following seven concrete contributions:

- An architectural paradigm—declarative governance over shared immutable context—that replaces predefined agent topologies with policy-driven coordination over a common semantic graph.

- A *bitemporal CRDT-inspired semantic graph* with append-only event log, dual valid-time and transaction-time axes, and monotonicity invariants that make point-in-time reconstruction, temporal contradiction detection, and regulatory audit structurally native.

- A product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ that unifies governance admissibility and convergence rank into a single well-founded descent domain, with a deterministic reduction kernel guaranteeing epoch termination under $\delta_{\min}$-discretisation (Theorem 1). See Remark 1 for the precise algebraic status.

- A formal convergence mechanism: a disagreement function $V(t)$ (Theorem 2 establishes it as a genuine Lyapunov function for the restricted CRDT system), with five independent gates (monotonicity, evidence coverage, oscillation detection, quiescence, minimum content) that provide layered guarantees against premature finality.

- A perpetual finality lifecycle in which certified checkpoints—not terminal decisions—model the ongoing nature of regulated knowledge, with Ed25519-signed certificates binding each decision to the exact policy version that produced it.

- A three-tier governance routing architecture (deterministic rules, oversight agent, full LLM) with circuit-breaker fallback, ensuring continuous governance without LLM availability.

- Validation on two scenarios—M&A due diligence and financial consolidation with dual temporality—demonstrating convergence, contradiction resolution, temporal restatement handling, and structured human review (test and benchmark details in Appendix).

For clarity, the following are explicit limits of the paper's claims:

What the paper does not demonstrate.

- *Statistical generality.* The Project Horizon validation is a single scenario with synthetic documents; no confidence intervals, hypothesis tests, or distributional claims are supported.
- *Byzantine fault tolerance.* All agents are assumed cooperative. Adversarial agents that inject false claims, manipulate confidence scores, or game the governance protocol are not addressed. Integration with Byzantine-tolerant consensus machinery [3, 13] remains future work.
- *Scalability beyond proof of concept.* Validation covers 50 claims, 5 documents, and 7 agents. Coordination dynamics at production scale (thousands of claims, hundreds of agents) are untested.
- *Machine-checked convergence proofs.* Convergence guarantees are validated empirically through benchmarks and unit tests, not by formal verification tools or proof assistants.
- *Real LLM stochasticity.* Convergence benchmarks use synthetic trajectories. The oscillation detection and trajectory quality mechanisms are designed for real-world stochasticity but have not been validated against it.
- *Protocol self-modification.* Governance rules are declarative but static within a deployment. The system does not implement governed self-modification of its own coordination protocol, as studied by de la Chica Rodriguez and Vera Díaz [5].
- *Multi-scope finality.* Cross-scope governance, hierarchical finality (parent scope depending on child scopes), and inter-scope certificate chains are designed but not validated.

These boundaries are revisited in detail in Section 12.

2 Related Work

2.1 Pipeline Orchestration

Workflow engines (Apache Airflow, Temporal, Prefect) implement the DAG model: define a static graph, schedule tasks in topological order, apply retry logic on failure. This model is effective for deterministic ETL pipelines but breaks when agents produce conflicting outputs, when new evidence appears mid-execution, or when tasks are interdependent in non-linear ways. None provide formal finality guarantees or governance-as-coordination.

2.2 Agent Coordination Frameworks

Recent frameworks enable multi-agent collaboration:

AutoGen [28] introduces conversable agents with role-based chat patterns. Coordination is conversation-driven: agents exchange messages in predefined patterns (sequential, group chat,

nested). However, coordination topology is hardcoded in Python, there is no shared persistent state between conversations, and no formal convergence mechanism.

CrewAI [18] organizes agents into crews with roles, goals, and tools. A hierarchical manager delegates tasks. Coordination follows a fixed process (sequential or hierarchical), with no mechanism for handling contradictions between agent outputs or tracking disagreement over time.

LangGraph [14] models agent coordination as state machines with explicit edges and conditional routing. This is closer to our approach—state is explicit and transitions are defined—but routing logic is embedded in code, not declarative policy. There is no immutable audit log, no CRDT semantics, and no formal convergence tracking.

All three frameworks assume that coordination topology can be predefined. Our system replaces topology with policy: agents reason independently over shared state, propose transitions, and governance rules determine what happens next.

Recent synthesis work also argues that communication topology should be treated as a first-class research object in LLM multi-agent systems rather than fixed engineering scaffolding [29]. Our architecture is aligned with that diagnosis, but moves one level deeper: it constrains shared-state evolution to a monotone governance projection so that consistency does not depend on a fixed interaction script.

2.3 Byzantine Consensus and Multi-Agent Coordination

The Byzantine consensus problem, formalized by Lamport, Shostak, and Pease [13], established that agreement among distributed processes is possible if and only if $n \geq 3f + 1$, where f is the number of Byzantine-faulty nodes. Practical Byzantine Fault Tolerance (PBFT) [3] made this feasible in partially synchronous systems with $O(n^2)$ message complexity. Ren et al. [24] surveyed consensus problems in multi-agent coordination, establishing the graph-theoretic foundations for state agreement under static and time-varying topologies. Zheng et al. [31] introduced confidence-weighted Byzantine fault tolerance (CP-WBFT), incorporating agent reliability into consensus—a direction relevant to LLM-based agents whose outputs carry inherent uncertainty. Mao et al. [17] extended the Byzantine Generals framework to practical multi-agent systems with the Imperfect Byzantine Generals Problem (IBGP), addressing local coordination without full global consensus.

Our system does not implement Byzantine consensus directly; agents are assumed cooperative. However, the semantic graph's monotonic constraints (confidence ratchets upward, contradictions are irreversible) provide partial robustness against downward manipulation, and the three-tier governance architecture limits the impact of any single agent's faulty output by requiring governance approval for all state transitions.

2.4 Self-Evolving Coordination Protocols

De la Chica Rodriguez and Vera Díaz [5] introduced Self-Evolving Coordination Protocols (SECP): coordination mechanisms that permit bounded, auditable self-modification of their own decision rules while preserving formal invariants. In a controlled feasibility study with six decision modules evaluating six Byzantine consensus proposals, they demonstrated that (i) non-scalar coordination rules can be synthesized by current AI models, (ii) a single governed modification increased proposal coverage by 50% while preserving declared invariants, and (iii) a systematic trade-off exists between protocol coverage (how many proposals are accepted) and evaluator autonomy (the capacity of individual modules to impose non-compensable objections).

Their work is complementary to ours in a precise sense. SECP addresses *protocol-level coordination*: how heterogeneous evaluator judgments are aggregated into accept/reject decisions, and how

that aggregation rule can evolve. Our system addresses *state-level coordination*: how agents reason over shared context, how disagreement is tracked formally, and how convergence is certified. SECP's explicit open questions—multi-iteration convergence dynamics, formal invariant preservation under repeated modification, and audit trail infrastructure—are directly addressed by our Lyapunov-based convergence tracking, five-gate finality mechanism, and bitemporal audit graph. Conversely, their non-scalar coordination with non-compensable objection rights and Byzantine fault tolerance assumptions address gaps we acknowledge: our convergence metric is scalar, our governance rules are static within a deployment, and we do not handle adversarial agents.

A unified architecture combining SECP's governed protocol evolution with our formal convergence tracking would enable multi-iteration protocol modification where each iteration's effect on the Lyapunov disagreement function $V(t)$ is measured, gates prevent premature adoption of harmful modifications, and the certificate chain records the protocol version under which each decision was made.

2.5 Consensus and Convergence Theory

Olfati-Saber and Murray [21] established the Lyapunov consensus framework for multi-agent systems, proving that monotonically decreasing disagreement functions guarantee convergence under mild assumptions. We adapt this to knowledge-work coordination, defining a weighted disagreement function over four semantic dimensions. The classical theory [16] provides the mathematical foundation: if a scalar function $V \geq 0$ is strictly decreasing along system trajectories and $V = 0$ only at the equilibrium, the system converges.

Ruan et al. [25] introduced the Aegean consensus protocol for multi-agent reasoning, which detects incremental quorum convergence across concurrent agent executions with provable refinement monotonicity—solution quality is guaranteed non-decreasing. We adapt their monotonicity principle to our finality evaluator: Gate A requires β consecutive non-decreasing rounds before declaring convergence.

Yazici et al. [30] study networked LLM agents in a DeGroot-style linear opinion model over diverse communication graphs (opinions extracted from message text). Disagreement decays exponentially and *transient* convergence rates align with classical graph theory through the second-largest eigenvalue of the combination matrix; however, the *limiting* consensus departs from DeGroot's network-centrality-weighted prediction—outcomes are largely insensitive to initial conditions and track topic and model bias. This cautions against equating "LLM consensus" with classical, centrality-optimal agreement even when mixing-time structure remains spectral.

2.6 CRDT Semantics and Monotonic State

Laddad et al. [12] demonstrated CRDT monotonic merge operations (Keep CALM and CRDT) that prevent state regression in distributed systems, and recent observation-driven coordination work for LLM agents reinforces the same monotonic-state design pressure [23]. Our semantic graph adopts this principle: confidence scores ratchet upward only, contradictions once marked are irreversible, and nodes are staled rather than deleted.

2.7 CALM and LVars

The closest theoretical lineage of our coordination model is the CALM/LVars tradition. CALM establishes that a distributed program admits a coordination-free consistent implementation iff it is monotone [9, 15]. LVars formalize the same idea in deterministic parallel programming via monotone writes and threshold reads [11]. In our architecture, inflationary deltas correspond to

monotone writes, and finality gates correspond to threshold-style reads over shared state. The mapping is direct but not trivial: we extend the classical semilattice setting to richer, conflict-aware ordered evidence with governance-stratified update producers and convergence guarantees under stochastic LLM perturbations.

2.8 Stigmergic Coordination

Dorigo, Bonabeau, and Theraulaz [7] formalized stigmergy as a distributed communication paradigm: agents coordinate indirectly through local modifications to their environment (pheromone trails in ant colonies) rather than through direct message passing. We apply this principle to convergence-driven agent routing: each agent's activation is gated by the per-dimension convergence pressure of the shared semantic graph, causing agents to concentrate on bottleneck dimensions without centralized scheduling.

2.9 Bitemporal Data Models

Snodgrass [26] established the bitemporal data model: every fact carries two independent time axes—valid time (when true in the world) and transaction time (when recorded by the system). This enables point-in-time reconstruction on either or both axes, a capability required by financial regulations (SOX, IFRS 9) but absent from existing agent frameworks. We apply bitemporal modeling to the semantic graph, enabling temporal contradiction detection and regulatory audit.

2.10 Rewriting Systems and Termination Theory

The reduction kernel's structure parallels classical term rewriting systems [1], where states correspond to terms, transitions to rewrite rules, and termination is established via well-founded orderings. Newman's Lemma [19] shows that local confluence plus termination implies global confluence—a result relevant to our partial confluence claim (Section 11). Unlike classical rewriting, our rewrite rules are dynamically proposed by agents and filtered by an order-valued governance layer; the kernel selects deterministically among admissible candidates. The novelty lies not in termination *per se* but in stratifying proposal generation (stochastic, LLM-driven) from reduction (deterministic, order-governed) and unifying governance admissibility with convergence rank in a product order.

2.11 Lattice-Based Governance and Information Flow

Lattice models are standard in information-flow security [2, 6] and access control, where a partial order on security labels governs permissible information transfers. Davey and Priestley [4] provide the algebraic foundations. Formal Concept Analysis (FCA) [8, 27] derives lattice structures from object-attribute incidence relations, producing concept lattices with a Galois connection that makes the intent/extent duality explicit. Our governance chain $\mathcal{L}$ (MASTER $\leq$ MITL $\leq$ YOLO, ordered by permissiveness) is structurally simple; the contribution lies in forming the product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ with the convergence rank $\mathcal{A}$, producing a unified descent domain where both safety (policy admissibility) and liveness (convergence progress) are enforced by a single well-founded ordering. This construction is algebraically elementary—$\mathcal{M}$ is a well-founded poset, not a lattice in the full algebraic sense (Remark 1)—but its application to multi-agent governance, unifying policy, convergence, and anti-gaming invariants in a single descent domain, appears structurally uncommon. A natural direction for future work is to derive this structure as a concept lattice [8] where agent contributions and governance constraints form a Galois connection, providing richer algebraic properties (formal concepts, attribute implications) suited to regulated environments.

2.12 Modal Logic and Fixed-Point Reasoning

The reduction semantics aligns naturally with Propositional Dynamic Logic (PDL) and fixed-point reasoning via the μ-calculus [10]. Modalities in our framework are interpreted only over committed reductions (governance-approved transitions), not over proposed transitions: exploration does not alter the Kripke structure, and only reductions define accessibility relations. The product order $\mathcal{M}$ constrains admissible transitions but does not alter the logical layer. This restriction strengthens semantic clarity and separates the non-determinism of LLM-driven proposal generation from the determinism of governance evaluation.

2.13 Policy Engines and Access Control

The XACML standard [20] defines combining algorithms for multi-policy evaluation (deny-overrides, first-applicable). Pang et al. [22] formalized Zanzibar-style relationship-based access control, implemented in OpenFGA. Our architecture replaces external policy engines with a native Rust *reduction kernel* compiled via napi-rs: all policy evaluation, lattice admissibility checking, and mode routing execute as deterministic Rust code within the Node.js process, eliminating the overhead and operational complexity of external policy runtimes while preserving XACML combining algorithms for multi-policy resolution and OpenFGA for fine-grained governance of document access, transition approval, and policy modification.

3 Problem Setting

This section formalizes the core abstractions of the system. Definitions follow the style of de la Chica Rodriguez and Vera Díaz [5] to facilitate cross-referencing between protocol-level and state-level coordination.

3.1 Shared Context Graph

Definition 1 (Semantic Graph)**.** *Let $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ be a directed labeled graph where $\mathcal{N}$ is a set of typed nodes and $\mathcal{E} \subseteq \mathcal{N} \times \mathcal{N} \times \mathcal{L}$ is a set of labeled edges. Node types are drawn from $\mathcal{T} = \{\textsc{Claim}, \textsc{Goal}, \textsc{Risk}, \textsc{Entity}\}$. Edge labels are drawn from $\mathcal{L} = \{\textsc{Supports}, \textsc{Contradicts}, \textsc{Resolves}, \textsc{Supersedes}\}$.*

Each node $n \in \mathcal{N}$ carries:

- *a confidence score $c(n) \in [0, 1]$,*
- *a valid-time interval $[vf(n), vt(n))$ (when the fact holds in the world),*
- *a transaction-time interval $[ra(n), sa(n))$ (when the system recorded and possibly superseded it),*
- *a source provenance $\sigma(n)$ referencing the originating document and agent.*

Definition 2 (Monotonicity Constraints)**.** *The graph $\mathcal{G}$ satisfies three monotonicity invariants:*

1. ***Confidence ratchet:*** *For any node n, if $c(n) = x$ at transaction time t_1, then for all $t_2 > t_1$ where n is not superseded, $c(n) \geq x$.*
2. ***Irreversible contradictions:*** *If $(n_1, n_2, \textsc{Contradicts}) \in \mathcal{E}$ at transaction time t, the edge persists for all $t' > t$. Resolution requires adding a new* Resolves *edge, not deleting the contradiction.*

3. ***Staling, not deletion:*** *No node or edge is removed from $\mathcal{G}$. Superseded nodes receive a finite $sa(n)$ timestamp; the current view filters to $sa(n) = \infty$.*

These invariants ensure that $\mathcal{G}$, restricted to the current view, evolves monotonically toward resolution.

3.2 Governance Order and Product Structure

Definition 3 (Governance Level Chain)**.** *Let $\mathcal{L} = \{\textsc{Master}, \textsc{Mitl}, \textsc{Yolo}\}$ be a totally ordered set (chain) of governance levels with the permissiveness ordering $\textsc{Master} \leq \textsc{Mitl} \leq \textsc{Yolo}$. Intuitively, $\textsc{Master}$ is the most restrictive (every drift signal can block), $\textsc{Yolo}$ is the most permissive (only policy rules block), and $\textsc{Mitl}$ routes to human review. As a finite chain, $\mathcal{L}$ is trivially a lattice with $\wedge = \min$ and $\vee = \max$.*

Definition 4 (Convergence Rank)**.** *Let $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ be a k-dimensional convergence rank space with the product partial order: $\mathbf{a} \leq \mathbf{a}'$ iff $a_i \leq a'_i$ for all $i \in \{1, \ldots, k\}$. In the reference implementation, $k = 4$ with dimensions: claim confidence, contradiction resolution, goal completion, and risk score inverse.*

Definition 5 (Product Order)**.** *The* governance-convergence product order *is the Cartesian product $\mathcal{M} = \mathcal{L} \times \mathcal{A}$, ordered componentwise [4]:*

$$(\ell, \mathbf{a}) \leq_{\mathcal{M}} (\ell', \mathbf{a}') \iff \ell \leq_{\mathcal{L}} \ell' \ \wedge \ \mathbf{a} \leq_{\mathcal{A}} \mathbf{a}'$$

A point $m = (\ell, \mathbf{a}) \in \mathcal{M}$ encodes both the governance regime and the convergence state. The partial order on $\mathcal{M}$ is well-founded (it is a product of a finite chain and a bounded subset of $\mathbb{R}^k$).

Remark 1 (Algebraic status of $\mathcal{M}$)**.** *We use the term "product order" rather than "product lattice" deliberately. While $\mathcal{L}$ is a chain (hence a lattice) and $\mathcal{A} = [0, 1]^k$ with the componentwise order is a complete lattice (with $\wedge = \min$, $\vee = \max$ per component), the implementation does* not *use the join or meet operations on $\mathcal{M}$. The reduction kernel uses only the partial order (Definition 6) to test admissibility of transitions—it never computes the least upper bound or greatest lower bound of two governance states. Formally, $\mathcal{M}$ is a* well-founded partially ordered set *(poset) used as a descent domain for epoch termination, not a lattice in the algebraic sense.*

This paper establishes the product order as a practical tool for governance admissibility. Companion work extends the stack with graph-structured evidence coordination among roles (developed separately). A natural extension—currently under investigation—is to derive $\mathcal{M}$ as a concept lattice *in the sense of Formal Concept Analysis [8, 27]: a structure where agent contributions form the* extent*, governance constraints form the* intent*, and the lattice order emerges from the Galois connection between them. Such a construction would provide richer structure (formal concepts, attribute implications, concept navigation) than the current product order, and connect naturally to the audit, lineage, and constraint-satisfaction requirements of regulated environments. This formalization is the subject of planned future work.*

Definition 6 (Admissibility)**.** *Given the current lattice point $m = (\ell, \mathbf{a})$ and a proposed point $m' = (\ell', \mathbf{a}')$, the* admissibility result *is:*

$$\mathrm{Adm}(m, m') = \begin{cases} \textsc{Admissible} & \text{if } \ell' \leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \leq_{\mathcal{A}} \mathbf{a} \\ \textsc{GovernanceViolation} & \text{if } \ell' \not\leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \leq_{\mathcal{A}} \mathbf{a} \\ \textsc{ConvergenceViolation} & \text{if } \ell' \leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \not\leq_{\mathcal{A}} \mathbf{a} \\ \textsc{BothViolated} & \text{if } \ell' \not\leq_{\mathcal{L}} \ell \ \wedge \ \mathbf{a}' \not\leq_{\mathcal{A}} \mathbf{a} \end{cases}$$

The four cases are exhaustive and mutually exclusive (the classification is a 2×2 grid on $\{\ell' \leq \ell, \ell' \not\leq \ell\} \times \{\mathbf{a}' \leq \mathbf{a}, \mathbf{a}' \not\leq \mathbf{a}\}$). Only ADMISSIBLE *transitions preserve descent in $\mathcal{M}$.*

3.3 Reduction Kernel

Definition 7 (Proposal)**.** *Let $\mathcal{A}_{agents} = \{a_1, \ldots, a_n\}$ be a set of agents. Each agent a_i observes the current view of $\mathcal{G}$ and emits a* proposal *$p = (\delta, j, a_i, \ell)$ where δ is a candidate state transition (a set of node/edge additions or confidence updates satisfying Definition 2), j is a natural-language justification, a_i is the proposing agent, and $\ell \in \mathcal{L}$ is the governance level.*

Definition 8 (Reduction Kernel)**.** *The* reduction kernel *$\mathcal{K}$ is a deterministic function:*

$$\mathcal{K} : \mathcal{P} \times \mathcal{G} \times \mathcal{R} \times \mathcal{M} \longrightarrow \{\textsc{Accept}, \textsc{Reject}, \textsc{Escalate}\} \times \Sigma$$

where $\mathcal{P}$ is the proposal space, $\mathcal{G}$ is the current graph state, $\mathcal{R}$ is the active rule set, $\mathcal{M}$ is the current lattice point, and Σ is the space of structured reasons (policy identifiers, regressed dimensions, suggested actions). The kernel evaluates a proposal p in three sequential stages:

1. ***Policy check.*** *Evaluate transition rules $\mathcal{R}$ against drift state: `canTransition`$(s, s', d, \mathcal{R})$ checks whether drift level d blocks the $s \to s'$ transition; `evaluateRules`$(d, \mathcal{R})$ computes suggested actions. If blocked:*
   - *$\ell = \textsc{Master} \implies \textsc{Reject}$ (most restrictive: policy blocks are final).*
   - *$\ell \in \{\textsc{Mitl}, \textsc{Yolo}\} \implies \textsc{Escalate}$ (route to human or oversight).*
2. ***Lattice admissibility.*** *Compute $\mathrm{Adm}(m, m')$ (Definition 6) between the current and proposed lattice points:*
   - *$\textsc{GovernanceViolation}$ or $\textsc{BothViolated} \implies \textsc{Reject}$.*
   - *$\textsc{ConvergenceViolation}$ or $\textsc{BothViolated}$: $\ell = \textsc{Master} \implies \textsc{Reject}$; otherwise $\textsc{Escalate}$.*
3. ***Mode routing.*** *If policy and lattice checks pass:*
   - *$\ell = \textsc{Mitl} \implies \textsc{Escalate}$ (human approval required).*
   - *$\ell \in \{\textsc{Master}, \textsc{Yolo}\} \implies \textsc{Accept}$ (reason: `policy_passed`).*

The central system invariant is: no agent directly modifies $\mathcal{G}$. *All modifications pass through $\mathcal{K}$.*

Theorem 1 (Epoch Termination Under Discrete Evaluation ($\delta_{\min}$-discretisation))**.** ***Precondition ($\delta_{\min}$-discretisation):*** *the implementation evaluates each convergence dimension at discrete intervals with a minimum step size $\delta_{\min} > 0$, so the effective rank space is $\mathcal{A}_\delta = (\delta_{\min}\mathbb{Z}_{\geq 0} \cap [0, 1])^k$, which is finite. ($\delta_{\min}$ is an implementation parameter exposed in finality configuration; it is not a mathematical property of $\mathcal{A}$ itself. Without this precondition $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ admits infinite descending chains and the theorem does not hold.)*

Under this precondition, within a single evidence epoch (no exogenous document injection), if the governance-approved transitions produced by $\mathcal{K}$ satisfy the monotonicity constraints of Definition 2, then the sequence of lattice points $m_0, m_1, \ldots$ is strictly descending in $\mathcal{M}_\delta = \mathcal{L} \times \mathcal{A}_\delta$ and terminates in finitely many steps.

Proof sketch. Each approved transition satisfies $m_{t+1} \leq_{\mathcal{M}} m_t$ by the admissibility check (Definition 6). Under the $\delta_{\min}$-discretisation precondition, $\mathcal{A}_\delta$ is finite: it contains at most $\lfloor 1/\delta_{\min} \rfloor^k$ points. $\mathcal{L}$ is finite (3 elements). Therefore $\mathcal{M}_\delta = \mathcal{L} \times \mathcal{A}_\delta$ is finite, and every strictly descending chain terminates. Strict descent requires at least one convergence dimension to make progress (Proposition 1); when no agent makes progress the sequence terminates at a fixed point (the stalled regime). In either case the sequence terminates in at most $|\mathcal{M}_\delta|$ steps. □

3.4 Convergence and Finality

Definition 9 (Disagreement Function)**.** *Let $\mathcal{D} = \{d_1, \ldots, d_k\}$ be a set of convergence dimensions, each with a weight $w_d > 0$ ($\sum_d w_d = 1$), a target value τ_d, and a measurement function $\mu_d : \mathcal{G} \to [0, 1]$. The disagreement function is (shown to be a genuine Lyapunov function for Σ_{CRDT} in Theorem 2):*

$$V(t) = \sum_{d \in \mathcal{D}} w_d \cdot (\tau_d - \mu_d(\mathcal{G}_t))^2$$

where $\mathcal{G}_t$ is the graph state at discrete time t (the t-th governance cycle). By construction, $V(t) \geq 0$, and $V(t) = 0$ if and only if all dimensions have reached their targets.

Definition 10 (Finality Predicate — Scalar Finality (Fallback))**.** *Let $S(t) = 1 - V(t)/V_{\max}$ be the normalized goal score. Define five gate predicates:*

$$G_A(t) = \bigwedge_{i=0}^{\beta-1} [S(t-i) \geq S(t-i-1) - \epsilon_m] \qquad \textit{(monotonicity, } \beta = 3,\ \epsilon_m = 0.001\textit{)}$$

$$G_B(t) = \mathit{EvidenceCoverage}(\mathcal{G}_t) \wedge (\mathit{ContradictionMass}(\mathcal{G}_t) = 0) \qquad \textit{(evidence + contradiction)}$$

$$G_C(t) = (Q(t) \geq 0.7) \qquad \textit{(trajectory quality, no oscillation)}$$

$$G_D(t) = (\mathit{IdleCycles}(t) \geq \gamma) \wedge (\mathit{IdleTime}(t) \geq \omega) \qquad \textit{(quiescence)}$$

$$G_E(t) = (|\mathcal{N}_t| > 0) \wedge (|\mathit{Goals}(\mathcal{G}_t)| > 0) \qquad \textit{(minimum content)}$$

The finality predicate is:

$$F(t) = [S(t) \geq \theta_{auto}] \wedge \bigwedge_{i \in \{A,B,C,D,E\}} G_i(t)$$

where $\theta_{auto} = 0.92$. When $F(t) = 1$, the system transitions to Resolved *and issues a signed finality certificate. When $F(t) = 0$ but $S(t) \geq \theta_{hitl}$, a human-in-the-loop review is triggered.*

Remark 2 (Scalar Finality as Fallback to Vector Finality)**.** *Definition 10 specifies the* scalar *finality predicate, where the normalized goal score $S(t)$ is compared against a single threshold (0.92). This predicate is now active only when per-dimension finality is disabled in configuration (backward-compatible fallback mode).*

When per-dimension finality is enabled (the default in production), Definition 12 (Section 5.4) takes precedence: the system evaluates $F^*(t)$ instead of $F(t)$, providing stronger guarantees against compensation attacks. Both predicates may be evaluated in parallel for diagnostic purposes (`compensation_detected` flag), but only $F^*(t)$ gates transitions to Resolved.

3.5 Threat Model and Assumptions

The current system operates under the following assumptions:

1. **Cooperative agents.** All agents are assumed to operate in good faith within their governance constraints. No Byzantine, adversarial, or colluding agents are modeled.

2. **Trusted governance layer.** The policy engine, epoch-based CAS, and finality evaluator are assumed correct. Formal machine-checked verification of these components is not provided.

3. **Trusted infrastructure.** Postgres, NATS, and OpenFGA are assumed to function correctly. Network partitions and service degradation are handled by circuit breakers and graceful degradation, not by Byzantine fault tolerance.

4. **Single-scope operation.** Cross-scope governance and hierarchical finality are designed but not validated.

These assumptions are substantially weaker than those required for Byzantine consensus ($n \geq 3f + 1$) [13]. Relaxing them—particularly the cooperative-agent assumption—is the primary direction for future work (Section 12).

3.6 Complexity Bounds

The three-node state cycle constrains the coordination topology. In each cycle (ContextIngested $\rightarrow$ FactsExtracted $\rightarrow$ DriftChecked), each of n agents is activated at most once (activation filters enforce this). The message complexity per cycle is therefore $O(n)$: one proposal per agent, one governance evaluation per proposal, one execution per approved proposal. Over k cycles to convergence, total message complexity is $O(nk)$.

This contrasts with classical BFT protocols requiring $O(n^2)$ messages per consensus round [3]. The reduced complexity follows from the cooperative-agent assumption: without Byzantine faults, the system does not need all-to-all message exchange for agreement. Instead, agreement is mediated by the shared graph $\mathcal{G}$ and the governance function Π.

The number of cycles k to convergence is bounded above by the EXPIRED timeout (configurable, default 30 days) and below by the minimum $\beta+\gamma$ rounds required by Gates A and D. In the Project Horizon validation, $k = 12$.

4 Core Design

4.1 Design Principle: Proposals, Approval, Execution

The central invariant of the system is: *no agent directly modifies shared state.* Instead:

1. **Agents propose:** Each agent reads shared context, reasons about it, and emits a proposal (a candidate state transition with justification).

2. **Kernel evaluates:** The reduction kernel $\mathcal{K}$ (Definition 8) evaluates the proposal against policy rules, lattice admissibility, and governance mode, producing a deterministic verdict.

3. **Executor implements:** Only approved proposals are applied to shared state, via atomic epoch-based compare-and-swap (CAS) transitions.

This pattern provides complete auditability: every state change has a proposal, an evaluation, and an execution record. It also enables governance without code changes: rules are configuration, not compiled logic.

4.2 Shared Immutable Context

All agents read from and write to two shared data structures:

Append-Only Event Log (Context WAL). Every claim, fact, and agent decision is recorded as an immutable event in a Postgres write-ahead log (example event schema in Appendix A). Events are never modified or deleted. This provides tamper-proof auditability, full reproducibility (replay the log to reconstruct any state), and causal traceability (which facts triggered which agent actions).

CRDT-Inspired Semantic Graph. A Postgres + pgvector database maintains semantic relationships with monotonic guarantees [12]:

- **Monotonic confidence:** Confidence scores ratchet upward only. A claim at 0.85 can become 0.92 but never revert to 0.80.
- **Irreversible contradictions:** Once a contradiction edge is created between two claims, it cannot be deleted—only resolved by creating a resolution edge.
- **Staling, not deletion:** Superseded nodes are marked stale, preserving the full reasoning history.

These CRDT semantics prevent state regression and enable formal convergence guarantees: the semantic graph can only move toward resolution, never away from it.

Bitemporal Semantic Graph. Beyond monotonic merge, the semantic graph implements *dual temporality* [26]: every node and edge carries two independent time axes.

- **Valid time** (`valid_from`, `valid_to`): the interval during which a fact holds in the real world. A revenue figure may be valid for fiscal year 2024; a compliance certificate valid until its expiry date.
- **Transaction time** (`recorded_at`, `superseded_at`): when the system learned or corrected the fact. A node superseded at t_s remains in the graph with `superseded_at` $= t_s$; its replacement carries `recorded_at` $= t_s$.

The *current view* filter—`superseded_at IS NULL` $\wedge$ (`valid_to IS NULL` $\vee$ `valid_to` $>$ `now()`)—restricts queries to facts that are both temporally valid and not yet corrected. As-of queries on either axis or both enable point-in-time reconstruction:

- **As-of valid time:** "What was believed true on reporting date T?"
- **As-of transaction time:** "What did the system know at audit date T'?"
- **Combined:** "What did the system know at T' about facts valid at T?"

Contradiction detection respects valid-time overlap: two claims contradict only if their validity windows intersect. A revenue figure valid in Q1 does not contradict a revised figure valid from Q2 onward. This prevents false contradictions from temporal misalignment and enables the finality evaluator to correctly assess unresolved disagreements.

An *evidence schema* declares required evidence types and maximum staleness per domain. Gate B of the finality evaluator (Section 5.6) uses this schema to block finality when required evidence is missing or has exceeded its temporal validity—ensuring that decisions are not finalized on stale data.

4.3 Declarative Governance

Agent activation and state transitions are defined in human-readable YAML (mode, drift-triggered rules, transition-blocking rules; see Appendix A).

Three governance modes correspond to levels in the governance lattice $\mathcal{L}$ (Definition 3):

- **YOLO** ($\ell = \textsc{Yolo}$, most permissive): Automatic approval for transitions that pass policy and lattice checks. Suitable for low-risk, internal workflows.
- **MITL** ($\ell = \textsc{Mitl}$, intermediate): All proposals that pass policy checks are routed to human review. Required for standard compliance workflows.
- **MASTER** ($\ell = \textsc{Master}$, most restrictive): Every proposal is evaluated through the full reduction kernel. Policy blocks produce Reject (not Escalate). Lattice incomparability produces Reject. Only transitions that are both policy-compliant and lattice-admissible are accepted. Suitable for highest-sensitivity scenarios where no ambiguity is tolerable.

Per-scope overrides allow mixing modes within a single system (e.g., YOLO for low-risk extraction, MITL for financial claims).

Reduction Kernel Architecture. Behind the YAML surface, a native Rust reduction kernel (compiled via napi-rs as a Node.js addon) evaluates all governance decisions deterministically. The kernel implements the three-stage pipeline of Definition 8: policy check (`canTransition` + `evaluateRules`), lattice admissibility (Definition 6), and mode routing. All pure mathematics—Lyapunov computation, gate evaluation, convergence analysis, and governance logic—executes in compiled Rust; TypeScript orchestrates I/O (database, message bus, YAML parsing). This separation ensures that the deterministic core is auditable, replayable, and verifiable independently of LLM stochasticity.

When multiple policy rules contribute to a single decision, XACML-inspired combining algorithms [20] resolve conflicts: `deny-overrides` (any deny wins) and `first-applicable` (ordered evaluation) are implemented. This mirrors the Policy Decision Point (PDP) / Policy Enforcement Point (PEP) architecture standard in enterprise access management.

Decision Records. Every governance evaluation produces an immutable `DecisionRecord` (schema in Appendix A) with `policy_version` binding. The `policy_version` is a content hash of the governance and finality configuration files at evaluation time, binding each decision to the exact rule set that produced it. This enables post-hoc queries such as: "Was this decision made under the pre-amendment or post-amendment compliance rules?"

4.4 Three-Tier Governance Routing

A critical design constraint for regulated environments is that governance must function even when LLM services are degraded or unavailable. The system implements three tiers of governance evaluation, each a strict superset of the previous:

1. **Deterministic evaluation (Tier 1).** Every proposal is first evaluated with zero LLM tokens: the reduction kernel $\mathcal{K}$ checks policy rules, lattice admissibility, and mode routing (Definition 8); OpenFGA checks agent permissions. This produces a deterministic outcome (Accept, Reject, or Escalate) with a structured reason. In MASTER mode or when no LLM is configured, this is the final decision.

2. **Oversight agent (Tier 2).** In YOLO mode with an LLM available, a lightweight oversight agent receives the deterministic result and chooses one of three actions: *accept* the deterministic result as-is, *escalate to full LLM* for richer reasoning, or *escalate to human* (MITL). This routing decision is itself audited via the `GovernancePath` field in the context WAL.

3. **Full governance agent (Tier 3).** When the oversight agent escalates, a full LLM-backed governance agent reasons over state, drift, governance rules, and policy checks using tool calls, then publishes an approval or rejection with natural-language rationale.

A *circuit breaker* (3 consecutive failures, 60-second cooldown) protects against LLM service degradation: when the breaker opens, Tiers 2 and 3 are bypassed and the system falls back to Tier 1 deterministic evaluation. This guarantees that governance never stalls due to LLM unavailability—a requirement in environments where agent downtime has compliance implications.

Every decision records which tier produced it (`processProposal`, `oversight_acceptDeterministic`, `oversight_escalateToLLM`, `oversight_escalateToHuman`, or `processProposalWithAgent`), providing auditors with the exact governance path for each state transition.

4.5 Three-Node State Cycle

Rather than an open-ended state machine, the system operates on a tight three-node cycle with epoch-based CAS:

$$\texttt{ContextIngested} \rightarrow \texttt{FactsExtracted} \rightarrow \texttt{DriftChecked} \rightarrow \texttt{ContextIngested}$$

Each transition requires governance approval. Only one agent succeeds per cycle (atomic epoch check). This prevents race conditions and ensures every cycle produces a complete audit record.

4.6 Fine-Grained Access Governance (OpenFGA)

OpenFGA [22] enforces relationship-based access control at four levels:

- **Document access:** Only authorized agents and users read sensitive materials.
- **Transition approval:** Governance rules determine who can approve which transitions.
- **Policy modification:** Changes to governance rules require role-based sign-off and version control.
- **Audit access:** Sensitivity-appropriate visibility into audit logs.

5 Convergence Theory and Finality

5.1 The Finality Problem

Traditional systems declare finality by threshold: "if confidence > 0.95, done." This is brittle: contradictions may emerge after the threshold is crossed, the system may cycle indefinitely, and there is no formal guarantee of stabilization. We replace threshold-based finality with *stateful convergence tracking.*

5.2 Disagreement Function and Restricted Dynamical System

We define a scalar *disagreement function* $V(t) \geq 0$ measuring distance to convergence targets (following the disagreement-function tradition of Olfati-Saber and Murray [21]):

$$V(t) = \sum_{d \in \mathcal{D}} w_d \cdot (\text{target}_d - \text{actual}_d(t))^2$$

Restricted dynamical system. The system operates under two distinct input regimes: governance-approved CRDT transitions, and exogenous document injection (evidence epochs). V behaves fundamentally differently in each. To ground a formal stability claim, we isolate the *restricted dynamical system* Σ_{CRDT}:

Definition 11 (Restricted CRDT System Σ_{CRDT})**.** *The restricted system Σ_{CRDT} is the closed-loop evolution of $\mathcal{G}_t$ in which:*

1. *all state transitions are governance-approved and satisfy the CRDT monotonicity invariants (Definition 2),*
2. *no new documents are injected (no cross-epoch events),*
3. *the facts agent introduces no new nodes beyond those already present in $\mathcal{G}_0$.*

Under Σ_{CRDT}, the only admissible state changes are: confidence ratchets, contradiction resolution edges, goal completion marks, and risk-node removals—all monotone in the direction of decreasing V.

Theorem 2 (Lyapunov Stability of Σ_{CRDT})**.** *Under Σ_{CRDT} (Definition 11), $V(t)$ is a Lyapunov function for the fixed point $\mathcal{G}^* = \{\mathcal{G} : V(\mathcal{G}) = 0\}$: it is non-negative by construction, equals zero if and only if all dimensions reach their targets, and is non-increasing along every trajectory of Σ_{CRDT}. When at least one dimension makes strict progress, the decrease is strict.*

Proof. Non-negativity is immediate from the squared-distance form. $V = 0$ iff $\mu_d = \tau_d$ for all d. Each transition in Σ_{CRDT} satisfies the monotonicity constraints of Definition 2 by precondition 1: $\mu_d(\mathcal{G}_{t+1}) \geq \mu_d(\mathcal{G}_t)$ for all d. Since $\tau_d \geq \mu_d(\mathcal{G}_t)$ when targets are not yet met, each squared term $w_d(\tau_d - \mu_d)^2$ is non-increasing, so $V(t+1) \leq V(t)$. Strict progress on any dimension with nonzero weight yields $V(t+1) < V(t)$. □

Full system behaviour. Outside Σ_{CRDT}—when cross-epoch document injection occurs—V may increase. This is an exogenous input to the closed-loop system, not a trajectory of Σ_{CRDT}, and does not contradict Theorem 2. The full system alternates between Σ_{CRDT} phases (where Theorem 2 applies) and injection events (where V may spike). Section 5.5 characterises both regimes and their interaction.

where $\mathcal{D}$ comprises four weighted dimensions (Table 1; note $\mu_4 = 1 - \text{risk}$ so that all dimensions are oriented toward 1):

Dimension	Notation	Weight	Target τ_d	Meaning
Claim confidence	$\mu_1 \in [0,1]$	0.30	≥ 0.85	Active claims at sufficient co
Contradiction resolution	$\mu_2 \in [0,1]$	0.30	$= 1.0$ ($\leq \epsilon_2$ residual)	Zero unresolved contradictio
Goal completion	$\mu_3 \in [0,1]$	0.25	≥ 0.90	Completion ratio of declared
Risk score inverse	$\mu_4 = 1 - \text{risk} \in [0,1]$	0.15	≥ 0.80	Residual risk below 0.20

Table 1: Convergence dimensions with weights and targets. Dimension 4 is defined as $\mu_4 = 1 - \text{risk}$; a target $\mu_4 \geq 0.80$ is equivalent to residual risk < 0.20. The same notation τ_d and μ_d is used throughout (Table 2).

5.3 Convergence Guarantee Under Monotonic Constraints

The CRDT-inspired monotonicity invariants (Definition 2) provide the foundation for a convergence argument. We state this as a proposition with a proof sketch; a fully machine-checked proof is deferred to future work.

Proposition 1 (Monotonic Progress Under CRDT Constraints — Intra-Epoch)**.** *Let $\mathcal{G}_t$ be the semantic graph at cycle t, and let $V(t)$ be the disagreement function (Definition 9).* Preconditions: *(i) no new documents are injected (single evidence epoch); (ii) every state change to $\mathcal{G}_t$ is a governance-approved transition satisfying the CRDT monotonicity invariants of Definition 2; (iii) no non-governance operations (e.g. facts-agent extraction runs) introduce new nodes or risk-tagged entities into $\mathcal{G}_t$ during the epoch. Under these preconditions, if at least one of the following holds:*

1. *a claim confidence $c(n)$ increases for some $n \in \mathcal{N}_t$,*
2. *a contradiction edge receives a* Resolves *edge,*
3. *a goal is marked complete,*
4. *a risk node is resolved or removed,*

then $V(t+1) \leq V(t)$, with strict inequality when the affected dimension has nonzero weight and the target has not yet been reached. Equivalently, under these preconditions V is a Lyapunov function for the restricted system Σ_{CRDT} (Definition 11).

Proof sketch. Each convergence dimension d contributes $w_d \cdot (\tau_d - \mu_d(\mathcal{G}_t))^2$ to $V(t)$. Under preconditions (i)–(iii), every state change is a governance-approved monotone write; no exogenous node can appear to raise V:

- *Claim confidence* (dimension 1): the confidence ratchet (Definition 2) guarantees $c(n)$ is non-decreasing, so the fraction of claims above threshold is non-decreasing.
- *Contradiction resolution* (dimension 2): contradictions are irreversible and resolutions are additive, so the unresolved count is non-increasing.
- *Goal completion* (dimension 3): goals are completed monotonically under governance-approved transitions; no new goal nodes enter (precondition iii).
- *Risk score inverse* (dimension 4): with no new risk-tagged nodes introduced (precondition iii), the risk score can only decrease as unresolved contradictions are resolved or risk nodes are cleared by approved transitions.

Hence $\mu_d(\mathcal{G}_{t+1}) \geq \mu_d(\mathcal{G}_t)$ for all d, the squared terms are non-increasing, and strict progress on any dimension with nonzero weight yields $V(t+1) < V(t)$.

Scope note. Precondition (iii) is violated in practice when the facts agent extracts new goals or risk-tagged entities from an arriving document within an epoch boundary. Such events are counted as exogenous shocks in the cross-epoch regime (Section 5.5) and do not contradict this proposition; they simply fall outside its preconditions. □

Rank function and bounded convergence. The per-dimension *rank function* $r(t) = \sum_{d \in \mathcal{D}} \max(0, \tau_d - \mu_d(\mathcal{G}_t))$ measures the total remaining gap to the per-dimension thresholds τ_d. Within a single evidence epoch, under the conditions of Proposition 1, $r(t)$ is non-negative and non-increasing at each cycle where at least one dimension makes progress; when no dimension makes progress, $r(t)$ is unchanged (stalled regime). Thus $r(t)$ provides a well-founded measure for intra-epoch convergence. Cross-epoch context injection (new documents, new goals or contradictions) can increase $r(t)$ before it resumes decreasing in the next epoch.

Two convergence regimes. The proposition above holds within a single evidence epoch—the period between consecutive document injections. When new evidence arrives (a *cross-epoch* event), it may introduce new contradictions or invalidate prior claims, causing V and $r(t)$ to increase. The system therefore operates in two regimes:

- **Intra-epoch** (V and $r(t)$ non-increasing): agents process existing evidence under monotonic constraints. $V(t)$ and $r(t)$ can only decrease. The progress indicator $\alpha_{\text{intra}} \geq 0$ measures progress within this regime (high variance of α across recent epochs indicates a non-stationary regime in which ETA estimates are unreliable).
- **Cross-epoch** (V may spike): new evidence enters the graph. The delta $\Delta V_{\text{cross}} = V(t_{\text{after}}) - V(t_{\text{before}})$ is typically positive when contradictions are introduced and negative when resolutions are provided. This is an exogenous input, not a governance failure.

The implementation tracks both regimes by tagging each convergence point with the WAL sequence number (`context_seq`) of the latest document. Pairs with the same `context_seq` contribute to α_{intra}; pairs where `context_seq` changes contribute to the cross-epoch delta.

Remark 3 (Operational Claim: Bounded Intra-Epoch Convergence)**.** *This is a* conditional operational claim*, not a theorem derivable from the preceding results alone. Its key precondition—"every governance cycle produces at least one approved transition making strict progress"—is a* liveness *condition that the architecture cannot guarantee: agents are LLM-driven and may generate proposals that are rejected or make no measurable progress for arbitrarily many cycles. The claim holds when and only when the liveness condition is met.*

Under the preconditions of Proposition 1 and *the additional liveness condition that every governance cycle produces at least one approved transition making strict progress on some dimension, $r(t) \to 0$ and $V(t) \to 0$ within an evidence epoch in at most K cycles. With $\delta_{\min}$-discretisation (Theorem 1), $K \leq \sum_{d \in \mathcal{D}} \tau_d / \delta_{\min}$.*

When the liveness condition is not met (stalled regime), the system falls back to plateau detection (Gate C) and human-in-the-loop routing rather than to this bound.

Empirical status. The liveness precondition is satisfied in Exp. 1: within evidence epochs where resolutions are provided, at least one dimension (`contradiction_resolution` or `goal_completion`)

makes strict progress per cycle, and $V(t)$ reaches 0.0000 at resolution epochs (epochs 9, 13, 17). Goal and risk nodes are protected from stale marking as accumulative types, allowing the resolver agent to advance `goal_completion` monotonically. Cross-epoch document injection introduces new goals that temporarily raise V, producing the sawtooth trajectory described below.

Perpetual lifecycle. In realistic deployments, the system alternates between intra-epoch convergence and cross-epoch evidence shocks. The resulting $V(t)$ trajectory is a *sawtooth*: decreasing within each epoch, spiking at evidence boundaries, then re-converging. This models the operational reality described in Section 5.9, where finality is a checkpoint, not a terminal state.

Connection to lattice-based termination. Proposition 1 establishes intra-epoch progress on the scalar $V(t)$ and on the rank function $r(t)$. The Epoch Termination Theorem (Theorem 1) strengthens this by proving that the product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5) provides a well-founded descent domain for the reduction kernel. Since $\mathcal{L}$ is finite (three governance levels) and $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ admits only finitely many admissible transitions within a bounded evidence epoch (the convergence rank can decrease at most finitely many times before reaching a fixed point), the combined ordering bounds the total number of governance-approved transitions per epoch.

This yields a two-level convergence guarantee: (i) within each evidence epoch, each admissible transition either preserves or decreases the lattice point in $\mathcal{M}$, and only finitely many such transitions are possible before the system reaches a fixed point (Theorem 1); (ii) each such transition satisfies the monotonicity constraints of Proposition 1, ensuring $V(t)$ is non-increasing. Together, these guarantee that the system reaches a stable governance configuration within each epoch in finite time.

The connection to rewriting theory [1] is structural: each governance-approved transition is a rewrite step, and the well-founded ordering on $\mathcal{M}$ serves as the termination measure. Newman's Lemma [19] would bridge local confluence to global confluence, but *local confluence is not established*: the current system guarantees that each individual governance evaluation is deterministic (the kernel $\mathcal{K}$ is a function), but it does not prove that two admissible transitions applied in either order yield joinable states. The system therefore provides *partial confluence*—certified compatible transitions commute—but full Church–Rosser confluence remains unproven and may be unrealistic in a governed multi-agent setting where proposals are dynamically generated. This framing also parallels Denning's lattice model [6], where information flow is bounded by a lattice ordering—here, governance permission flow is bounded by $\mathcal{L}$, and convergence progress is bounded by $\mathcal{A}$.

Note that intra-epoch convergence does *not* hold when agents fail to make progress (the stalled regime, $\alpha_{\text{intra}} \approx 0$). Stalled-dimension detection identifies dimensions where the per-dimension score has not improved in the last 5 intra-epoch evaluations. Plateau detection (Gate C) and human-in-the-loop routing provide operational safeguards for this case. The proposition also does not address adversarial agents—a limitation addressed by the threat model (Section 3).

5.4 Per-Dimension Finality and Non-Compensability

The convergence framework maintains CRDT-inspired monotonicity on each convergence dimension (Definition 9). However, the scalar finality predicate (Definition 10) permits a subtle vulnerability: one dimension can over-improve (e.g., claim confidence $\mu_1(t) = 1.0$) to compensate for another dimension's failure (e.g., unresolved contradictions $\mu_2(t) = 0.80$), causing the normalized score $S(t)$ to exceed threshold even when epistemic safety is compromised. This is the *compensation artifact*: scalar-pass / vector-fail states where the system would declare Resolved despite residual uncertainty.

To address this, we introduce *vector finality*, a per-dimension predicate that enforces independent safety thresholds on each dimension while preserving the global gate structure.

5.4.1 Vector Finality Predicate Definition

Definition 12 (Vector Finality Predicate)**.** *Let $\mathcal{D} = \{d_1, \ldots, d_k\}$ be the set of convergence dimensions (claim confidence, contradiction resolution, goal completion, risk score inverse). For each dimension $d \in \mathcal{D}$, define:*

- τ_d *— per-dimension threshold*
- ϵ_d *— per-dimension epsilon tolerance*
- $e_d(t) = \max(0, \tau_d - \mu_d(\mathcal{G}_t))$ *— per-dimension finality gap*
- $VETO_d$ *— boolean flag indicating whether dimension d blocks finality unconditionally*

The vector finality predicate is:

$$F^*(t) = \bigwedge_{d \in \mathcal{D}} \Big[e_d(t) \leq \epsilon_d \wedge G_A^d(t) \wedge G_C^d(t) \Big] \wedge \bigwedge_{i \in \{B,D,E\}} G_i(t)$$

where $G_A^d(t)$ and $G_C^d(t)$ are per-dimension variants of gates A and C (monotonicity and trajectory quality), and $G_B(t), G_D(t), G_E(t)$ remain global (evidence coverage, quiescence, and minimum content).

When $VETO_d$ = true for some dimension d, a failure of the threshold check $e_d(t) \leq \epsilon_d$ blocks finality regardless of other dimensions' performance. The system transitions to Resolved *only when $F^*(t) = 1$.*

Relationship to $V(t)$. The Lyapunov function $V(t)$ tracks convergence dynamics; the finality predicates $F(t)$ and $F^*(t)$ are finality preconditions. $V(t) \to 0$ is necessary but not sufficient for finality (gates may block); $F^*(t) =$ true is necessary but not sufficient for $V(t) = 0$ (the threshold may be below the current $V(t)$).

5.4.2 Dimension-Specific Thresholds and Tolerances

Table 2 specifies the per-dimension parameters. Each dimension has been validated empirically to reflect the epistemic safety requirement it represents:

Dimension	τ_d	ϵ_d	**Interpretation**	**Veto?**
Claim confidence	0.85	0.02	Active claims at sufficient confidence	No
Contradiction resolution	0.95	0.01	Unresolved contradictions near zero	**Yes**
Goal completion	0.90	0.02	Declared goals substantially completed	No
Risk score inverse	0.80	0.03	Residual risk below operational threshold	No

Table 2: Per-dimension finality thresholds (τ_d), epsilon tolerances (ϵ_d), and veto semantics. The Veto column indicates whether a dimension blocks finality when it fails. Contradiction resolution is veto because unresolved contradictions represent irreducible epistemic uncertainty that cannot be compensated by high confidence or goal completion.

Veto Semantics. In the formal specification, all required dimensions participate in the conjunction. However, when a dimension is marked veto, its failure automatically causes $F^*(t) = \text{false}$ regardless of other dimensions' scores. This ensures that epistemic safety requirements (particularly contradiction resolution) cannot be overridden by compensatory improvement in other dimensions. The veto flag is configuration-driven (see finality.yaml per-dimension-finality section) and can be extended as operational experience evolves.

5.4.3 Gate Refactoring: Per-Dimension vs. Global Gates

Gate	Scope	Description
G_A^d	Per-dimension	Score $\mu_d(t)$ non-decreasing for last $\beta = 3$ cycles within dimension d
G_C^d	Per-dimension	Trajectory quality $Q^d(t) \geq 0.7$ (lag-1 autocorrelation) for dimension d
G_B	Global	Evidence coverage > 0 and contradiction mass $= 0$ across entire scope
G_D	Global	Quiescence: idle cycles $\geq \gamma$ and idle time $\geq \omega$
G_E	Global	Minimum content: $\lvert\mathcal{N}_t\rvert > 0$ and $\lvert\text{Goals}(\mathcal{G}_t)\rvert > 0$

Table 3: Gate refactoring under vector finality. Gates A and C now evaluate per-dimension properties (monotonicity and trajectory quality on each dimension separately), while gates B, D, E remain global because they characterize the scope as a whole.

Monotonicity per dimension (Gate G_A^d). Rather than requiring the normalized score $S(t)$ to be non-decreasing (original scalar Gate A), each dimension d must have non-decreasing $\mu_d(t)$ over the last β cycles. This prevents false finality from oscillation: a dimension cannot spike up, then drop, and still pass finality.

Trajectory quality per dimension (Gate G_C^d). The lag-1 autocorrelation of dimension d's score history must be ≥ 0.7, indicating a stable upward trajectory with minimal oscillation within that dimension. This complements Gate A by detecting subtle noise that monotonicity alone might miss.

5.4.4 Proof Obligation PO-2: Non-Compensability

Theorem 3 (Non-Compensability: Vulnerability Existence and Vector Finality Prevention)**.** *The theorem has two parts.*

Part 1 (Compensation vulnerability exists in scalar finality). *There exist reachable system states in which the scalar finality condition $F(t) = \text{true}$ holds (i.e. $S(t) \geq \theta_{auto} = 0.92$) while the system has not achieved epistemic safety on all dimensions (i.e. $\exists\, d : e_d(t) > \epsilon_d$). Such states are reachable whenever one dimension over-improves to compensate for another's shortfall. This constitutes a structural vulnerability of scalar-finality systems.*

Proof by construction. *Take $\mu = (1.0,\ 0.80,\ 1.0,\ 1.0)$. The scalar score is $0.30 \cdot 1.0 + 0.30 \cdot 0.80 + 0.25 \cdot 1.0 + 0.15 \cdot 1.0 = 0.94 \geq 0.92$, so $F(t) = \text{true}$. But the contradiction resolution dimension has gap $e_2(t) = 0.95 - 0.80 = 0.15 > \epsilon_2 = 0.01$, meaning 20% of contradictions remain*

unresolved. This state is reachable via legitimate LLM-driven confidence inflation; the test case `vector_finality_blocks_compensation` *constructs it explicitly.*

Part 2 (Vector finality closes the vulnerability). *Under the CRDT monotonicity assumption (A3), the vector finality predicate $F^*(t)$ (Definition 12) guarantees that if $F^*(t) = \text{true}$ then for all required dimensions d:*

1. *$e_d(t) \leq \epsilon_d$ (each dimension gap is within tolerance),*
2. *$G_A^d(t) = \text{true}$ (per-dimension monotonicity over the last β cycles),*
3. *$G_C^d(t) = \text{true}$ (per-dimension trajectory quality ≥ 0.7),*
4. *if $\text{VETO}_d = \text{true}$, then $e_d(t) > \epsilon_d \Rightarrow F^*(t) = \text{false}$ (veto dimensions block finality unconditionally).*

Part 1's compensation state is blocked: $e_2(t) = 0.15 > \epsilon_2 = 0.01$ and $\text{VETO}_{contradiction} = \text{true}$ together force $F^*(t) = \text{false}$.

Proof. *Part 2 follows directly from the definition of F^* as a conjunction: $F^*(t)$ is false whenever any conjunct fails. The CRDT assumption guarantees μ_d is non-decreasing, so once $e_d(t) \leq \epsilon_d$ is achieved it persists; the gate conditions G_A^d, G_C^d are explicit predicates evaluated at declaration time.*

5.4.5 Backward Compatibility and Fallback

The vector finality predicate $F^*(t)$ is controlled by a configuration flag (`per_dimension_finality.enabled` in finality.yaml). When disabled, the system falls back to the original scalar predicate $F(t)$ (Definition 10), preserving backward compatibility. This allows staged rollout and comparative evaluation (see experiments below).

The dual-path implementation in the finality evaluator (Section 5.4) ensures that:

1. When vector finality is enabled, $F^*(t)$ is the authoritative predicate; $F(t)$ is computed but not used for finality decisions.
2. When vector finality is disabled, $F(t)$ is used (legacy behavior).
3. A diagnostic flag `compensation_detected` is logged whenever a state satisfies $F(t)$ but not $F^*(t)$, enabling empirical measurement of compensation artifacts.

5.5 Convergence Rate and ETA

The instantaneous progress indicator is $\alpha(t) = -\ln(V(t)/V(t-1))$, defined for $V(t-1) > 0$ and $V(t) > 0$. At full convergence ($V(t) = 0$), α is not computed; the system reports ETA = 0 and transitions to RESOLVED via the finality predicate rather than via the α path. Two aggregated values are maintained:

- **Mixed** α (backward compatible): averaged over the last 5 consecutive pairs regardless of evidence boundaries. Used for divergence detection ($\alpha < -0.05$ triggers ESCALATED).
- **Intra-epoch** $\alpha_{\textbf{intra}}$: averaged only over pairs where `context_seq` has not changed. Measures genuine agent progress, excluding evidence injection shocks. Used for ETA computation.

The estimated time to arrival uses α_{intra} and is defined for $V(t) > 0$ and $\alpha_{\text{intra}} > 0$:

$$\text{ETA} = \left\lceil \frac{-\ln(\epsilon/V(t))}{\alpha_{\text{intra}}} \right\rceil, \quad \epsilon = 0.005, \quad V(t) > 0$$

ETA is reported as null when $\alpha_{\text{intra}} \leq 0$ (no intra-epoch progress) or when $V(t) = 0$ (convergence already reached), preventing the misleading estimates that arise when cross-epoch spikes average α to near-zero.

Three regimes:

- $\alpha_{\text{intra}} > 0$: Converging within current evidence. Finite ETA.
- $\alpha_{\text{intra}} \approx 0$: Stalled. Stalled-dimension detection identifies which dimensions lack progress. Plateau detection (Gate C) activates.
- Mixed $\alpha < -0.05$: Diverging (possibly due to cross-epoch evidence shock). Triggers ESCALATED state.

5.6 Finality Gates

Auto-finality (RESOLVED) requires passing all five gates simultaneously. Each gate addresses a distinct failure mode of premature finality.

Gate A: Monotonicity [25]. Goal score must remain non-decreasing for $\beta = 3$ consecutive rounds before auto-finality eligibility. A single drop > 0.001 resets the counter. This prevents declaring convergence during transient spikes.

Gate B: Evidence Coverage and Contradiction Mass. An evidence schema (Section 4.2) declares required evidence types per domain and maximum staleness (`max_age_days`). Gate B blocks finality when required evidence types are missing or when evidence has exceeded its temporal validity. Contradiction mass—the weighted count of unresolved contradictions—must also be zero for auto-resolution. This ensures that finality reflects substantive coverage, not merely the absence of detected problems.

Gate C: Oscillation Detection and Trajectory Quality. Gate C addresses a subtle failure mode: a system whose goal score oscillates around the finality threshold without genuinely converging. Two statistical mechanisms detect this:

1. **Direction-change counting:** In a sliding window of the last 10 goal-score evaluations, the number of direction reversals (positive-to-negative or vice versa, with a 0.001 dead band) is counted. Two or more reversals flag oscillation.
2. **Lag-1 autocorrelation:** The Pearson correlation between the goal-score series and its one-step lag is computed. Negative autocorrelation ($r_1 < -0.3$) confirms alternating behavior characteristic of oscillation.

A *trajectory quality score* $Q \in [0, 1]$ synthesizes both signals:

$$Q = \max(0,\ 1 - \lambda \cdot \min(\text{direction_changes}, k))$$

with further reduction to $Q \leq c_{\text{osc}}$ when $r_1 < \theta_r$ and to $Q \leq c_{\text{spike}}$ on spike-and-drop (latest score well below window maximum). Auto-RESOLVED requires $Q \geq 0.7$. Default parameters: $\lambda = 0.12$, $k = 5$, $\theta_r = -0.3$, $c_{\text{osc}} = 0.65$, $c_{\text{spike}} = 0.85$. All five constants are exposed as configuration parameters.

Sensitivity analysis. A sweep of ±20% on each of the five Q constants across 7 benchmark scenarios (70 evaluations) found that only `spike_drop_cap` (c_{spike}) is sensitive: reducing it by 20% flips Gate C from pass to fail in 2 of 14 evaluations (the spike-and-drop and divergence scenarios). All other constants are robust to ±20% variation. Total gate flips: 2/70 (2.9%). This indicates that Q is operationally stable, with c_{spike} as the only constant requiring domain-specific calibration.

Gate D: Quiescence. Configurable via `idle_cycles_min` and `window_ms` in finality configuration. When enabled, RESOLVED is blocked unless the scope has been idle (no state transitions) for the configured number of cycles and time window. This prevents premature finality during active processing bursts where the score happens to cross the threshold momentarily.

Gate E: Minimum Content. When all dimensions score 1.0 vacuously—because there are zero claims, zero goals, and zero risks in the graph—Gate E blocks auto-finality. An empty or trivially-seeded scope cannot be declared resolved. This prevents the degenerate case where a newly initialized scope immediately satisfies all threshold conditions.

Pressure-Directed Activation [7]. Agents are routed toward the dimension with highest residual gap (bottleneck), without centralized scheduling. This stigmergic mechanism ensures resources concentrate where they matter most.

Divergence Escalation. If $\alpha < -0.05$ (disagreement increasing), the system transitions to ESCALATED state, requiring human intervention.

Human-in-the-Loop Review. When goal score is in $[0.40, 0.92)$ and plateau is detected, the system queues a structured HITL review with: dimension breakdown, blocking factors, LLM-generated explanation, and suggested actions (approve finality, provide resolution, escalate, or defer).

5.7 Finality States

State	Trigger	Action
RESOLVED	Vector finality $F^*(t)$ true (all dims $\geq \tau_d - \epsilon_d$, gates A–E)	JWS certificate issued
ACTIVE	Score < 0.92 or any gate unsatisfied	Continue processing
ESCALATED	Risk ≥ 0.75 or ≥ 3 contradictions or $\alpha < -0.05$	Human intervention
BLOCKED	≥ 5 idle cycles + ≥ 300s without updates + ≥ 1 contradiction	Stalled
EXPIRED	No activity for 30 days	Process expired
HITL Review	Score $\in [0.40, 0.92)$ + plateau or gate failure	Structured human decision

Table 4: Finality states and their triggers. RESOLVED requires vector finality $F^*(t)$ (all dimensions within threshold tolerance, per-dimension gates $\mathrm{A}^d/\mathrm{C}^d$, global gates B/D/E per Definition 12). Upon RESOLVED, an Ed25519-signed finality certificate is emitted (Section 5.8). Fallback to scalar finality $F(t)$ (Definition 10) is available when per-dimension finality is disabled.

5.8 Finality Certificates

When finality is reached—either through automatic convergence (all gates passed) or human approval—the system issues a cryptographically signed *finality certificate* providing non-repudiable proof of the decision.

The certificate payload contains:

```
{
  "scope_id": "project-horizon",
  "decision": "RESOLVED",
  "timestamp": "2025-08-15T14:22:00Z",
  "policy_version_hashes": {
    "governance": "sha256:7e4f2a...",
    "finality": "sha256:b3c91d..."
  },
  "dimensions_snapshot": {
    "claim_confidence": 0.94,
    "contradiction_resolution": 1.0,
    "goal_completion": 0.92,
    "risk_score_inverse": 0.88
  }
}
```

This payload is signed as a compact JWS (JSON Web Signature) using Ed25519 (EdDSA), producing a base64url-encoded triple: `header.payload.signature`. Verification requires only the public key and no system access, enabling external auditors, counterparties, or regulators to independently validate that:

1. A specific scope reached a specific finality decision;
2. The decision was made at a specific timestamp;
3. The governance and finality rules in effect were identified by content hashes, binding the decision to an exact, reproducible rule set;
4. The dimensional scores at the moment of finality are recorded.

Certificates are persisted in a dedicated `finality_certificates` table and exposed via API for retrieval. Key management supports both environment-provided PEM keys (production) and ephemeral key pairs (development).

For regulated environments, finality certificates serve as machine-verifiable sign-off records—the agent-system equivalent of an authorized signatory's approval, with the added property that the policy version under which the decision was made is cryptographically bound to the certificate.

5.8.1 Experimental Validation of Vector Finality

Three experiments (exp-ab, exp8-compensate, Exp 9 sub-test 7) validate the non-compensability guarantee (PO-2): vector finality blocks scalar-pass/vector-fail states while preserving legitimate finality. The witness `vector_finality_blocks_compensation` ($\mu = (1.0, 0.80, 1.0, 1.0)$) confirms $F(t) = \text{true}$ but $F^*(t) = \text{false}$. Test counts and protocol details are in Appendix B.

5.9 Perpetual Finality: From Checkpoint to Checkpoint

A critical distinction separates this architecture from systems that treat finality as terminal. In most regulated domains, knowledge does not reach a permanent resting state. New documents arrive, regulations are amended, market conditions shift, periodic reviews are mandated, and previously resolved contradictions may re-emerge under new evidence. Finality here is not the end of a process—it is a *certified checkpoint* within an indefinite lifecycle.

The shared context graph enables this directly. When a scope reaches RESOLVED:

1. A finality certificate is issued and persisted, recording the decision, the dimensional scores, the policy version, and the timestamp.
2. The semantic graph is *not archived or frozen*. It remains the live, authoritative representation of knowledge for that scope.
3. When new context arrives—a new document ingested into the WAL, a regulatory change requiring re-evaluation, or a scheduled periodic review trigger—the scope re-enters ACTIVE state.
4. The new convergence cycle starts from the *existing graph state*: all prior claims, contradictions, resolutions, and confidence scores are preserved. New facts layer on top. New contradictions may form between new and old claims.
5. The Lyapunov function $V(t)$ is recomputed. If new contradictions have increased disagreement, V rises and convergence must recur. If the new information merely confirms or refines prior conclusions, V remains low and finality may be reached quickly.
6. A new finality certificate is issued when convergence is re-achieved, creating a *chain of certificates*: each certifies the scope's state at a specific point in time, under a specific policy version, with a specific dimensional snapshot.

Bitemporal state is what makes this work without information loss. Prior facts are never deleted. When a new document supersedes a previous claim (e.g., restated financials), the old node receives a `superseded_at` timestamp; the new node carries its own `valid_from`. The full temporal history is preserved: an auditor can reconstruct what the graph looked like at any prior finality point, compare it to the current state, and see exactly which new information triggered the re-evaluation.

This architecture models the operational reality of regulated knowledge work: M&A post-merger monitoring, ongoing KYC/AML reviews, and pharmacovigilance surveillance all follow this pattern—initial finality followed by periodic re-convergence on the same graph as new evidence arrives. The three-node cycle runs indefinitely; what changes between cycles is the graph's content, not its structure.

6 Agent Architecture

Seven agents (facts, drift, planner, status, governance, executor, tuner) operate independently within governance constraints. Each is gated by deterministic activation filters (sequence_delta, hash_delta, timer, pressure_directed) that consume zero LLM tokens and prevent redundant invocations. Agents do not call each other directly; they emit events that trigger governance rules, enabling resilience, full auditability, and independent scaling. Per-agent roles and activation logic are in Appendix C.

7 Comparison with Agent Frameworks

Property	AutoGen	CrewAI	LangGraph	SECP	Ours
Coordination model	Chat	Hierarchical	State machine	Protocol agg.	Decl. policy
Shared persistent state	No	No	Partial	No	Yes
Bitemporal state	No	No	No	No	Yes
Formal convergence	No	No	No	No†	Yes (Lyapunov)
Contradiction tracking	No	No	No	Implicit‡	Yes (graph)
Immutable audit log	No	No	No	Partial	Yes
Non-scalar coordination	No	No	No	Yes	Yes*
Protocol self-modification	No	No	No	Yes	No
Byzantine fault model	No	No	No	Yes	No
Declarative governance	No	No	No	No	Yes (YAML+Rust)
Cryptographic finality	No	No	No	No	Yes (Ed25519)
Fine-grained access control	No	No	No	No	Yes (OpenFGA)
CRDT monotonic semantics	No	No	No	No	Yes
HITL finality	Manual	Manual	Manual	No	Structured
Agent topology	Predefined	Predefined	Predefined	Fixed	Topology-free††
LLM-free fallback	No	No	No	N/A	Yes (Tier 1)

Table 5: Comparison with agent coordination frameworks and SECP [5]. †SECP demonstrates single-step modification but not multi-step convergence; Exp. 1 provides multi-iteration data. ‡Module disagreement is tracked through non-scalar objections but not in a persistent graph. *The vector finality predicate $F^{*}(t)$ (Section 5.4) evaluates each dimension independently with veto semantics, implementing the non-compensable objection property SECP identifies as essential (Theorem 3, PO-2). ††Agents have no peer-to-peer communication channels; coordination arises from shared state and governance evaluation, not from an agent-to-agent communication graph.

The table reveals two orthogonal gaps. Existing agent frameworks wire agents into topologies with no formal convergence, auditability, or protocol adaptability. SECP addresses protocol-level coordination (non-scalar aggregation, governed self-modification, Byzantine fault model) but lacks shared persistent state or formal convergence tracking. Our system addresses state-level coordination with formal convergence and non-scalar finality, but does not support protocol self-modification. Three of SECP's explicitly stated open questions—multi-iteration convergence dynamics, audit trail infrastructure, and coverage–autonomy quantification—are directly addressed by experiments in this paper (Exps. 1, 5, 7, 9 and the certificate infrastructure). Conversely, SECP's non-compensable objection mechanism addresses our acknowledged gap on adversarial agents. The complementarity is analyzed in Section 11.

8 Validation: Project Horizon

8.1 Scenario and Results

Project Horizon simulates M&A due diligence on NovaTech AG (B2B SaaS, supply chain compliance). Five documents are ingested sequentially (analyst briefing, financial DD, technical assessment, market intelligence, legal review), introducing facts and contradictions. Governance blocks prevent silent advancement on the EUR 12M ARR discrepancy; medium drift escalates to risk review; near-finality triggers structured HITL with dimension breakdown. Key outcome: 18 contradictions identified, 15 agent-resolved, 3 routed to human; 12 convergence rounds, full audit trail.

Detailed scenario steps and metrics are in Appendix D.

8.2 Testing and Benchmarks

Test breakdown and benchmark scenarios are in Appendix B.

8.3 Known Validation Gaps

The initial validation (above) uses synthetic convergence benchmarks, not live LLM trajectories. Section 10 reports experimental results from the full pipeline with live LLM extraction, addressing several of these gaps. A second validation scenario (financial consolidation with dual temporality) broadens domain coverage. Remaining gaps: no stress testing for high throughput, no chaos engineering for service failures, and unmeasured LLM oversight quality (the oversight agent consistently agreed with deterministic results).

9 Experimental Protocol

The following experiments address the open questions identified in this work and in the SECP feasibility study [5]. They are specified with sufficient detail for independent reproduction. Together, they target five axes: convergence dynamics, scalability, finality robustness, multi-level governance, and the coverage–autonomy trade-off. Results are reported in Section 10.

9.1 Experiment 1: Multi-Iteration Convergence Dynamics

Objective. Characterize the trajectory of $V(t)$ over extended convergence cycles with incremental evidence injection—the multi-iteration dynamics that SECP [5] explicitly could not study with their single-shot design.

Protocol.

1. Initialize a scope with a seed document producing ∼20 claims.
2. Over 20 convergence cycles, inject one new document per cycle, each introducing $c \in \{0, 1, 3, 5\}$ contradictions with existing claims (controlled variable).
3. Record at each cycle t: $V(t)$, $\alpha(t)$, $S(t)$, gate satisfaction vector $(G_A, G_B, G_C, G_D, G_E)$, finality state, and the set of unresolved contradictions.
4. When finality is reached, continue injecting evidence to trigger re-opening (perpetual lifecycle).
5. Repeat each condition 10 times with different random seeds to estimate variability.

9.2 Experiment 2: Scalability

Objective. Provide the first empirical data on how governed agent coordination scales, addressing SECP's open question on scaling beyond small module sets.

Protocol.

1. Vary graph size: $|\mathcal{N}| \in \{10, 50, 100, 500, 1000\}$ claims.
2. Vary contradiction density: $\rho \in \{0.10, 0.30, 0.50\}$ (fraction of claim pairs that contradict).
3. Vary agent count: $|\mathcal{A}| \in \{3, 5, 7, 12\}$.
4. Fix governance mode to YOLO and finality thresholds to defaults.
5. Measure: rounds to convergence k, wall-clock time, total LLM token consumption, audit event count, semantic graph query latency.

9.3 Experiment 3: Finality Robustness Under Adversarial Evidence

Objective. Validate that the five-gate mechanism prevents false finality under evidence patterns designed to exploit threshold-based systems.

Protocol. Inject four adversarial evidence patterns, each designed to trigger a specific failure mode:

1. **Spike-and-drop:** Inject a batch of high-confidence confirmatory evidence (pushing $S(t)$ above θ_{auto}), immediately followed by contradictory evidence in the next cycle.
2. **Oscillating claims:** Alternate between confirmatory and contradictory evidence every cycle for 20 cycles.
3. **Stale evidence:** Allow evidence to age past `max_age_days` during a slow convergence process.
4. **Empty-scope finality:** Initialize a scope with zero claims and zero goals, then add a single high-confidence claim.

9.4 Experiment 4: Multi-Level Governance

Objective. Demonstrate governance at multiple bounded levels, extending both this work and SECP's single-level coordination model.

Protocol. Define three governance levels with increasing authority:

- **Level 1 (Operational):** YOLO mode. Handles routine claim extraction and low-drift transitions autonomously.
- **Level 2 (Compliance):** MITL mode. Activated when drift exceeds medium threshold or when financial claims are involved.
- **Level 3 (Regulatory):** MASTER mode. Activated for critical contradictions (e.g., material financial discrepancies, legal liability). Decisions at this level are immutable and cannot be overridden by L1 or L2.

Run the Project Horizon M&A scenario with per-scope governance level overrides: financial claims at L2, patent disputes at L3, technical assessments at L1.

9.5 Experiment 5: Coverage–Autonomy Trade-off

Objective. Empirically map the coverage–autonomy trade-off identified by SECP [5], using governance modes as the control variable.

Protocol.

1. Prepare a fixed document corpus (10 documents, ∼100 claims, 30 contradictions).
2. Run identical corpus through three governance configurations: YOLO, MITL (with simulated human approvals), and MASTER.
3. Define coverage $\Delta(\Pi)$ as the number of claims reaching RESOLVED status.
4. Define autonomy $\Omega(\Pi)$ as the fraction of governance decisions where a single agent's objection (drift signal) blocked or escalated a proposal.

Expected outcomes. YOLO should maximize coverage, MASTER should maximize evaluator autonomy (policy violations and lattice incomparability produce rejections), and MITL should occupy an intermediate position. The convergence rate indicator α should differ: $\alpha_{\text{YOLO}} > \alpha_{\text{MITL}} \geq \alpha_{\text{MASTER}}$.

10 Experimental Results

The experiments were executed with a document-driven driver through the full pipeline; results from PostgreSQL convergence history and decision records. Infrastructure: Docker (PostgreSQL/pgvector, NATS, MinIO, OpenTelemetry), single hatchery, OpenAI GPT-4o-mini. Full result tables are in Appendix E.

Experiment numbering. The protocol section specifies Experiments 1–5. Experiments 6 and 7 address governance-assumption validation: Exp. 6 validates the $\delta_{\min}$-discretisation precondition (Assumption A1) and monotonic progress (A4) via a synthetic trajectory sweep; Exp. 7 validates three-tier routing coverage across governance modes (A7). Both are reported in the Appendix (Assumptions A1–A7) rather than in a dedicated results subsection to avoid redundancy with the five primary experiments. Experiments 8 and 9 address adversarial agent resilience and local confluence; they extend the scope of the original five-experiment protocol based on reviewer input during development.

10.1 Experiment 1: Convergence Dynamics

With $c = 3$ contradicting documents and progressive resolution at rounds 5–7, $V(t)$ exhibits the predicted *sawtooth*: $0.250 \to 0.0000$ at resolution epochs (9, 13, 17), then 0.008–0.022 as new documents add goals. Full convergence ($V = 0$, score $= 1.0$) is reached at each resolution epoch before the next cross-epoch shock—confirming the perpetual finality lifecycle. The scalar $V(t)$ is driven by `goal_completion`; other dimensions saturate at 1.00.

10.2 Experiment 3: Finality Robustness (Spike-and-Drop)

Spike-and-drop (2 high-confidence then 2 contradicting documents): with gates enabled, Gate A blocks re-entry to RESOLVED after the drop; system moves to HITL. With gates disabled, ESCALATED falls to 0% and persistent RESOLVED rises (38% vs 33%), confirming that gates prevent false finality on transient peaks.

10.3 Experiment 2: Scalability

For $N \in \{10, 50, 100\}$, $\rho \in \{0.1, 0.3\}$: at $N = 10$–50, convergence points and epochs are similar; at $N = 100$, convergence points and decisions roughly double. Higher ρ yields fewer approved decisions (governance blocks). Scaling effects appear around $N = 100$.

10.4 Experiment 5: Coverage–Autonomy

YOLO, MITL, and MASTER were run on the same 7-document corpus. Result: YOLO maximizes coverage; MASTER rejects on policy/lattice violations (evaluator autonomy); MITL escalates to human. Progress indicator α obeys $\alpha_{\text{YOLO}} > \alpha_{\text{MITL}} \geq \alpha_{\text{MASTER}}$, as predicted.

10.5 Experiment 4: Multi-Level Governance

Medium-drift variant: oversight LLM consistently agreed with Tier 1; Tier 2/3 paths not triggered in this run.

10.6 Pressure-Directed Activation

With pressure-directed routing disabled, ACTIVE share rises (63% vs 1%) and convergence points drop (27 vs 94), confirming that pressure routing concentrates evaluations on finality-relevant cycles.

10.7 Experiment 7: Multi-Tier Governance Routing (Assumption 4)

Same M&A corpus under YOLO, MITL, MASTER: YOLO 97% Tier 1; MITL 100% Tier 2 (human); MASTER 70% Tier 1 with 30% rejections on policy. Tier 1 and Tier 2 are exercised; Tier 3 was not triggered (oversight LLM agreed with deterministic result). Assumption 4 partially validated.

10.8 Experiment 8: Adversarial Agent Defense (Assumption 5)

Inflate (1 compromised agent) and collude (2 compromised) modes: drift agent raised overrides in inflate; in collude, false RESOLVED windows appeared but cycle-based re-extraction flushes mutations in 1–2 cycles, limiting impact. Assumption 5 partially validated: cooperative model is necessary; re-extraction provides unintended defense-in-depth.

10.9 Experiment 9: Local Confluence (Assumption 2)

Six sub-tests (confidence ratchet, idempotency, CRDT commutativity, eventual consistency, kernel determinism, cross-epoch): core CRDT operations are commutative; claim stale marking is order-dependent but all orderings converge after complete re-extraction. Kernel: 80/80 identical outputs. Assumption 2 partially validated.

10.10 Financial Consolidation with Dual Temporality

Meridian Holdings (3 subsidiaries, H1 2025): eight documents including a temporal restatement (Alpha Q1 restated, same valid period, later transaction time). $V(t)$ sawtooth ($0.25 \to 0.00$ at resolution epochs 24, 28–30, 36 $\to 0.25$ on new goals) matches Exp. 1. Valid-time overlap and goal-aware stale protection behave as designed. Domain-independent convergence confirmed; 45 convergence points over 38 epochs.

11 Discussion

11.1 What the Experiments Demonstrate

The experimental results provide evidence for four of the paper's central claims, while revealing important limitations in two others.

1. $V(t)$ tracks disagreement and converges to zero on resolution (confirmed). Across all conditions with contradictions, $V(t)$ increased reproducibly when contradicting documents entered the graph and *decreased to 0.0000* when resolutions were provided. In Exp. 1 ($c = 3$, resolution at rounds 5–7), V dropped from 0.250 to 0.0000 at resolution epochs, with goal score reaching 1.0000—full convergence. This pattern replicated across experiments: $V = 0$ was achieved in Exp. 1, 6, 7, 8, and the financial scenario. Cross-epoch document injection then re-introduces new goals, producing the sawtooth trajectory predicted by the perpetual lifecycle model. The Lyapunov function tracks genuine epistemic state bidirectionally: increasing with contradictions and new goals, decreasing to zero with resolutions.

2. Five-gate mechanism prevents false finality (confirmed). Exp. 3 demonstrates that the system reaches RESOLVED (score = 1.0) during the spike phase, then correctly exits RESOLVED when contradictions arrive. Gate A (monotonicity) and Gate B (evidence coverage) are the primary defenses: Gate B satisfaction drops from 100% to 58% when contradictions appear. The ablation (gates disabled) shows that without gates, the system is more permissive (RESOLVED 38% vs 33%, ESCALATED 0% vs 7%), confirming that gates provide an additional safety layer beyond threshold-based checking.

3. Governance mode affects coverage (confirmed). Exp. 5 demonstrates the coverage–autonomy trade-off across governance modes. YOLO processes all documents (23 decisions). MITL occupies an intermediate position with human-escalated review (40 decisions, all escalated). MASTER evaluates all proposals through the full reduction kernel, applying the three-stage pipeline (policy, lattice, mode routing) and only accepting proposals that pass both policy and lattice admissibility checks. Policy violations and lattice incomparability produce Reject, not Escalate. This maps directly to the lattice ordering Master $\leq$ Mitl $\leq$ Yolo in Definition 3 and aligns with SECP's finding that stricter evaluation reduces coverage.

4. Pressure-directed routing affects system dynamics (directional evidence). The pressure ablation shows a substantial difference in finality state distribution: pressure-enabled runs produce 94 convergence points with HITL dominating (84%), while pressure-disabled runs produce 27 points with ACTIVE dominating (63%). This suggests that pressure routing accelerates the system toward finality evaluation, though the single-run ablation design ($n = 1$) limits statistical confidence.

5. Multi-tier governance routing is exercised (confirmed). Exp. 7 demonstrates that the three governance modes produce observably different tier coverage: YOLO routes 97% of decisions through Tier 1, MITL escalates 100% to Tier 2 (human-in-the-loop), and MASTER rejects 30% of proposals at Tier 1 due to policy violations. Combined across modes, Tier 1 and Tier 2 are both exercised. This confirms that the lattice ordering Master $\leq$ Mitl $\leq$ Yolo produces structurally distinct governance behavior, not merely different labels on the same path.

6. Adversarial agents cause oscillating—not permanent—false finality (partially confirmed). Exp. 8 demonstrates that adversarial confidence inflation (single compromised agent) produces 3× more governance overrides than baseline (9 vs 3). With coordinated adversarial agents (facts + drift), the system oscillated between RESOLVED and HITL over 31 epochs (vs 18 for baseline), with 18 convergence snapshots reaching false RESOLVED. Cycle-based re-extraction flushed adversarial mutations within 1–2 cycles each time, preventing sustained false finality. The cooperative agent model (Assumption 5) is structurally necessary (adversarially induced RESOLVED windows exist, $V(t)$ monotonicity is violated), but the system has unexpected defense-in-depth: source re-extraction acts as a practical Byzantine defense.

7. Partial confluence holds for CRDT operations (confirmed). Exp. 9 demonstrates that the core semantic graph operations—confidence ratchet (max join), contradiction resolution (set-once), node upsert (content-matched)—are fully commutative. Claim stale marking is non-commutative (18/24 orderings diverge) but eventually consistent after complete re-extraction; goals and risks are protected from stale marking as accumulative types, narrowing the non-commutativity surface. The governance kernel is a pure deterministic function (80 evaluations, all identical). This validates the paper's claim of "partial confluence" and identifies claim stale marking as the specific operation preventing full commutativity.

11.2 What Remains Unproven

1. Full $V \to 0$ achieved within evidence epochs; cross-epoch oscillation persists. With goal-aware stale protection and progressive resolution (rounds 5–7), $V(t)$ reached 0.0000 at resolution epochs (epochs 9, 13, 17 in Exp. 1), with goal score reaching 1.0000 and all four dimensions at target. This is the first empirical confirmation that $V \to 0$ is achievable. However, cross-epoch document injection re-introduces new goals and contradictions, causing V to rise again (to 0.0078–0.0216). The overall trajectory is a *sawtooth*: convergence to 0 within each evidence epoch, followed by a cross-epoch shock when new documents arrive.

The intra/cross-epoch analysis reveals a subtlety: the regime boundary of Proposition 1 is finer than document injection. Facts-worker extraction can introduce new goals or contradictions *within* an evidence epoch—a non-governance operation that occurs alongside governance-approved transitions. The proposition's precondition ("governance-approved transitions") is formally correct, but the `context_seq` proxy includes non-governance operations. A finer-grained partition—separating governance-approved transitions from facts-extraction side effects—would be needed to test the proposition's intra-epoch guarantee directly.

Per-dimension analysis of the *post-resolution cross-epoch regime* (epochs 10–28) shows that oscillation in that phase is driven primarily by `goal_completion`: subsequent documents in this corpus introduce new goals that are not yet resolved, raising V from 0.0000 to 0.008–0.022. In that regime, `claim_confidence`, `contradiction_resolution`, and `risk_score_inverse` are near 1.00 because the post-resolution documents happen to be confirmatory—introducing no new contradictions and no low-confidence claims. This is a property of the five-document experimental corpus, not of the architecture: in a corpus where each new document simultaneously introduces contradictions, restates financial figures, and adds risk items (a realistic regulated-domain scenario), all four dimensions would oscillate cross-epoch. The perpetual lifecycle mechanism is designed for exactly that case.

In the *initial phase* (epochs 1–4), the picture is different: 18 contradictions enter and are unresolved, which necessarily depresses `contradiction_resolution` and contributes to $V = 0.250$. The isolation of `goal_completion` as the sole driver of cross-epoch oscillation is therefore specific to the

post-resolution regime in this corpus. The scalar metric then collapses this per-dimension dynamic into apparent oscillation between two values—masking whether the oscillation is one-dimensional (as here) or multi-dimensional (as in richer corpora). This is one concrete motivation for the per-dimension finality predicate $F^*(t)$: it surfaces per-dimension gaps that the scalar V obscures.

The architecture partially addresses this limitation through the product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5), where the convergence rank $\mathcal{A} = (\mathbb{R}_{\geq 0})^k$ tracks each dimension independently. The reduction kernel's admissibility check (Definition 6) evaluates per-dimension regression rather than relying solely on the scalar aggregate, preventing transitions that improve the aggregate while regressing individual dimensions. However, the finality evaluator's ultimate decision still relies on the scalar goal score and gate predicates. A fully per-dimension finality criterion—requiring each dimension to independently satisfy its target—remains a direction for future work.

2. Multi-path governance (partially observed). In Exp. 4, all decision records used `governance_path = oversight_acceptDeterministic`—the oversight LLM always agreed with the deterministic policy result. Exp. 7 addressed this limitation by running each governance mode (YOLO, MITL, MASTER) as a separate sub-experiment with unified policy rules. MITL mode produced 100% Tier 2 escalation (all proposals required human approval), confirming that the routing architecture functions correctly. However, Tier 3 (full LLM governance via the oversight agent choosing `escalateToLLM`) was still not triggered: the oversight LLM consistently accepted the deterministic result even under medium and high drift.

Two structural causes are identified. First, the oversight agent's routing instructions treated triggered obligations (`open_investigation`) as advisory rather than prescriptive, biasing toward `acceptDeterministic` even when policy had flagged a concern. Second, the M&A demo corpus produces clean deterministic outcomes with no genuine ambiguity between policy evaluation and context: there is no proposal that policy allows but context should question. Triggering Tier 3 requires proposals where the deterministic check returns "approve" but an obligation is active and the stakes are material—precisely the zone where LLM judgment adds value over rule application. A dedicated corpus (`docs-tier3/`) is designed for this purpose: six documents covering two contradictory risk assessments (€2.1M vs. €10.3M litigation exposure), a probabilistic IP title opinion (65/30/5% outcome distribution), and a final governance proposal explicitly in the "allowed under uncertainty" zone. This design targets the `open_investigation` obligation at medium drift with material financial stakes, which the oversight instructions now treat as a mandatory `escalateToLLM` signal. Exp. 7b (Tier-3 activation validation) is identified as the immediate next experimental step (Section 12).

3. Scalability at $N = 500$. Exp. 2 covers $N \in \{10, 50, 100\}$ via the full document-injection pipeline. To extend the scalability boundary, two additional runs at $N = 500$ were conducted using the seed-based protocol (direct graph construction, isolating the convergence and finality machinery from facts-extraction latency): $\rho = 0.1$ (50 contradiction pairs) and $\rho = 0.3$ (150 contradiction pairs). In both conditions, $V(t)$ began at 0.251 and fell to **0.001** (≈ 0) after one resolution cycle, confirming convergence at this scale. Resolving $\rho = 0.1$ required a single batch (50 edges); $\rho = 0.3$ required two passes (103 then 47 edges, 150 total). Following resolution, three injected documents re-introduced contradictions and raised V to ≈ 0.46—reproducing the sawtooth pattern of Exp. 1 at 50× the claim count. Evidence propagation at $\rho = 0.3$ required 63 internal epochs, consistent with an $O(N \cdot \rho)$ scaling of the resolution workload. Wall-clock time from seed to convergence was approximately 2 minutes for both ρ values. The Lyapunov convergence machinery therefore scales to $N = 500$ without qualitative change; the limiting factor is resolution throughput (batch size

and scheduling), not contradiction detection. Testing at $N = 1000+$ under concurrent document injection, and characterising the full throughput curve, remains open.

11.3 Threats to Validity

Construct validity. The Lyapunov function $V(t)$ is a designed metric, not a ground-truth measure of knowledge quality. A decrease in V indicates movement toward configured thresholds, not necessarily toward correct knowledge. The experiments measure V's response to document injection, not its correlation with epistemic accuracy.

Internal validity. The document corpus is synthetic (though realistic). Contradictions are explicit and clearly stated, not subtle or ambiguous. The noisy corpus (available but not yet tested in batch mode) was designed to address this gap. LLM stochasticity is partially controlled by $n = 3$ repetitions, but larger n would increase confidence.

External validity. Experiments use two domains: M&A due diligence and financial consolidation. The financial scenario confirms that convergence dynamics ($V(t)$ spike-and-recovery) are domain-independent. Generalization to additional regulated domains (KYC/AML, clinical trials) is asserted by architectural argument (Appendix G) but not empirically validated beyond these two.

Statistical power. With $n = 3$ runs per condition, the standard deviations are informative (e.g., V std $= 0.001$–0.015) but formal hypothesis testing (e.g., paired t-tests) would require larger samples. The current results should be interpreted as reproducibility evidence, not as statistically powered claims.

11.4 Implications for Future Work

The experimental results identify four specific research directions, ordered by theoretical impact:

Per-dimension finality (beyond order admissibility). The product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5) tracks each convergence dimension independently during governance evaluation, and the reduction kernel rejects transitions that regress any single dimension. However, the finality evaluator's ultimate decision still relies on a scalar goal score and gate predicates. Extending the finality gates to require per-dimension satisfaction would: (a) prevent declaring finality when individual dimensions have not reached their targets; (b) expose per-dimension convergence rates as first-class diagnostic signals; (c) provide the per-dimension α_d needed for meaningful ETA estimation; (d) align with SECP's non-scalar coordination, where Pareto filtering and constructive objections prevent the collapse of structured disagreement. This extension is straightforward within the ordering framework: finality requires $a_d \leq \epsilon_d$ for each dimension d in the convergence rank, rather than a single scalar threshold.

Graph-structured evidence and the monotonicity–fidelity tension. Companion work extends the architecture with role-local evidence coordinated over communication graphs; it suggests a sharp tension between statistically faithful multi-agent aggregation and order-theoretic monotonicity of a live working estimate—motivating deliberate *separation* between a convergence-oriented layer and an append-only audit trail. Independent large-scale opinion-dynamics experiments on networked LLMs [30] exhibit the same split at the phenomenological level: transient mixing remains spectral (second eigenvalue), yet limiting consensus need not match classical DeGroot-style,

centrality-weighted predictions. Formal statements, mechanisms, and validation scope are deferred to the companion paper so that this manuscript stays self-contained.

Autonomous goal resolution. The resolver agent cross-references evidence and produces resolves edges for contradictions. Goal-aware stale protection allows `goal_completion` to advance when goals are resolved by the experiment driver or resolver agent. The remaining gap is a dedicated goal-resolution agent that transitions goals to resolved status autonomously when supporting evidence reaches sufficient confidence—removing the dependency on the experiment driver's `injectResolution()` mechanism and closing the convergence loop end-to-end within a single evidence epoch.

Finer-grained epoch boundaries. The `context_seq` proxy for evidence epochs includes non-governance operations (facts extraction, drift detection) alongside governance-approved transitions. A finer partition—tracking the exact governance transition that modifies the graph—would enable direct testing of the proposition's intra-epoch guarantee.

Production-scale and multi-domain validation. Testing at $N > 500$ documents, with noisy/ambiguous corpora, and across multiple regulated domains (KYC/AML, clinical trials) would establish external validity.

12 Scope, Limitations, and Future Work

Following the principled scoping methodology of de la Chica Rodriguez and Vera Díaz [5], we state explicitly and without mitigation rhetoric what this work establishes and what it does not.

12.1 What the Paper Demonstrates

1. **Architectural feasibility.** Declarative governance over shared immutable context is implementable with current technology (Postgres, NATS, LLMs, OpenFGA, native Rust via napi-rs) and produces a functioning coordination system; test and benchmark counts are reported in the Appendix.

2. **Formal convergence tracking with empirical $V \to 0$.** The disagreement function $V(t)$ (Theorem 2) with five independent gates prevents premature finality in all tested scenarios and reaches $V = 0.0000$ at resolution epochs—the first empirical confirmation that the convergence mechanism achieves its theoretical target. Proposition 1 provides a proof sketch for monotonic progress under CRDT constraints, now empirically confirmed within evidence epochs.

3. **Product order governance.** The product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ (Definition 5) provides a well-founded ordering over governance decisions, and the Epoch Termination Theorem (Theorem 1) guarantees finite convergence within evidence epochs. The reduction kernel (Definition 8) implements this ordering as a deterministic three-stage pipeline in compiled Rust.

4. **Perpetual lifecycle.** The architecture supports re-opening finality on new evidence without information loss, producing a chain of signed certificates that models the perpetual nature of regulated knowledge.

5. **Three-tier governance resilience.** The system continues to make governance decisions without LLM availability, falling back to deterministic rule evaluation via the circuit-breaker mechanism.

6. **End-to-end validation across two domains.** The Project Horizon M&A scenario demonstrates 83% autonomous contradiction resolution across 5 contradictory documents in 12 convergence rounds. A second scenario—financial consolidation with bitemporal restatement—confirms that the same spike-and-recovery dynamics hold when dual temporality is structurally necessary, with valid-time-scoped contradiction detection and temporal supersession operating correctly across 42 epochs.

7. **Coverage–autonomy trade-off.** The three governance modes (YOLO, MITL, MASTER) provide distinct positions on the coverage–autonomy spectrum identified by [5]: YOLO maximizes coverage (analogous to scalar aggregation), MASTER maximizes evaluator autonomy through the reduction kernel (policy violations and lattice incomparability produce rejections), and MITL provides an intermediate, auditable regime with human escalation.

12.2 What the Paper Does Not Demonstrate

1. **Statistical generality or significance.** Project Horizon is a single scenario with synthetic documents. No confidence intervals, hypothesis tests, or distributional claims are supported. Results are existence proofs, not performance estimates.

2. **Formal Byzantine fault tolerance.** All agents are assumed cooperative. Exp. 8 demonstrates that adversarial agents can cause oscillating false finality (18 RESOLVED snapshots across 31 epochs with coordinated collusion), but cycle-based re-extraction limits each adversarial window to 1–2 extraction cycles. This is a practical defense, not a formal Byzantine guarantee: the system provides no proven $n \geq 3f + 1$ tolerance. The monotonic confidence constraint limits downward manipulation but does not prevent upward inflation. Formal Byzantine-tolerant coordination [3, 13, 31] remains future work, though the empirical resilience is stronger than previously claimed.

3. **Scalability beyond proof of concept.** Validation covers two scenarios (5 and 8 documents, ∼50 claims each, 7 agents). Whether convergence dynamics, governance throughput, or the Lyapunov function's behavior change qualitatively at production scale (thousands of claims, hundreds of agents) is unknown.

4. **Machine-checked convergence proofs.** Proposition 1 is a proof sketch validated empirically. No formal verification tool or proof assistant has checked the convergence guarantee. An arbitrary sequence of approved transitions could, in principle, violate the proposition's preconditions in untested scenarios.

5. **Convergence under real LLM stochasticity.** All convergence benchmarks use synthetic trajectories. Whether the oscillation detection (Gate C) and trajectory quality score Q perform correctly under the inherent stochasticity of real LLM reasoning traces is untested; Q is domain-configurable with no formal justification (Appendix F).

6. **Multi-iteration protocol self-modification.** Governance rules are declarative but static within a deployment. The system does not implement governed self-modification of its own

coordination rules, as demonstrated by SECP [5]. Whether the Lyapunov convergence mechanism can serve as a formal foundation for evaluating protocol modifications across multiple iterations is a promising but unvalidated direction.

7. **Multi-scope and hierarchical finality.** Cross-scope governance, parent–child finality dependencies, and inter-scope certificate chains are designed in the architecture but not validated experimentally. A *multi-temporal lattice tower*—unifying governance, convergence, evidence, and temporal structure in one algebraic story—is under active investigation. The motivating hunch, consistent with both external LLM consensus evidence [30] and companion architectural work, is that *monotone* audit-friendly records and *faithful* multi-agent numerical agreement may require distinct mechanisms rather than a single protocol. Whether a product-style construction preserves the well-foundedness needed for epoch termination across layers remains open; details are reserved for follow-on publications.

8. **Human-in-the-loop effectiveness.** The HITL review mechanism is designed and implemented but not empirically evaluated with real human reviewers.

9. **Coverage–autonomy formalization.** While the governance lattice $\mathcal{L}$ (Definition 3) provides a formal ordering of governance strictness, and the reduction kernel implements distinct routing per level, no formal metric quantifies evaluator autonomy as a continuous variable. The lattice provides a qualitative ordering ($\textsc{Master} \leq \textsc{Mitl} \leq \textsc{Yolo}$); a quantitative measure (e.g., the probability that a single agent's objection blocks finality) and its systematic variation across governance configurations would provide a principled basis for mode selection.

Any claim beyond the narrow statement—"the architecture is implementable, produces auditable convergence in a controlled scenario, and provides formal safeguards against premature finality"—is unsupported by the evidence presented. Implementation and scalability limitations, theoretical limitations, the formal assumption matrix (A1–A7), future work directions, and the formal hardening summary (vector finality $F^*(t)$, PO-2) are collected in Appendix F.

13 Conclusion

13.1 Summary of Contributions

This paper presents a paradigm for autonomous agent coordination based on declarative governance over shared immutable context. The seven core contributions are: (i) a bitemporal CRDT-inspired semantic graph with append-only event log, dual time axes, and monotonicity invariants; (ii) a disagreement function $V(t)$ that is a genuine Lyapunov function for the restricted CRDT system (Σ_{CRDT}), with five independent gates providing layered guarantees against premature finality; (iii) a product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ with an Epoch Termination Theorem under $\delta_{\min}$-discretisation guaranteeing finite convergence via well-founded descent; (iv) a deterministic Rust reduction kernel ensuring auditable, replayable governance independent of LLM stochasticity; (v) perpetual finality with Ed25519-signed certificate chains; (vi) three-tier governance routing with LLM-free fallback; and (vii) validation on two domains—M&A due diligence and financial consolidation with bitemporal restatement—demonstrating $V(t) \to 0$ at resolution epochs and domain-independent sawtooth dynamics across evidence epochs.

13.2 Positioning Relative to Self-Evolving Coordination

This work and the Self-Evolving Coordination Protocol (SECP) study [5] address complementary halves of a shared problem. Where SECP demonstrates that bounded, governed self-modification of coordination protocols is technically feasible in a single-shot design, we demonstrate that formal convergence tracking over shared state provides the multi-iteration foundation such mechanisms need for sustained deployment. Specifically:

- SECP's open question on multi-iteration dynamics is addressed by our Lyapunov function $V(t)$ and the Epoch Termination Theorem (Theorem 1), which together guarantee finite convergence within each evidence epoch and track progress over indefinite cycles with formal safeguards against oscillation and premature finality.
- SECP's open question on audit infrastructure is addressed by our bitemporal graph and policy-version-bound certificate chain, which provide the temporal auditability their architecture currently checks empirically.
- The scalar convergence limitation is now fully addressed. The product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ tracks convergence per-dimension during governance evaluation, and the vector finality predicate $F^*(t)$ (Section F) extends this to the finality decision itself, requiring each dimension to independently satisfy its threshold with per-dimension gates. This implements the non-scalar convergence criterion that SECP's Pareto filtering and constructive-objection requirements identify as essential.
- Our open question on adversarial agents is partially addressed by Exp. 8, which demonstrates that cycle-based re-extraction provides practical (not formal) Byzantine resilience, and by SECP's non-compensable objection rights, which provide a complementary formal mechanism.

A synthesis—governed protocol evolution evaluated by Lyapunov convergence, with the product order providing per-dimension governance guarantees within each cycle and formal finality gates across cycles—would address the remaining limitations of both approaches. The proposed experimental protocol (Section 9) is designed to build toward this synthesis.

13.3 Interpretation Boundaries

The empirical claims are narrow and intentionally circumscribed (Sections 12 and 11). Results from $n = 3$ repeated runs demonstrate reproducibility of $V(t)$ response to contradictions and gate mechanism effectiveness, but are not statistically powered distributional claims. The convergence guarantee (Proposition 1) and the Epoch Termination Theorem (Theorem 1) hold formally and were empirically confirmed: $V(t) = 0.0000$ was achieved at resolution epochs in Exp. 1, 2, 3, 7, and 8, and across three distinct convergence cycles in the financial consolidation scenario. Five formal assumptions were experimentally validated: discretization and monotonic progress (Exp. 6), multi-tier governance routing (Exp. 7), cooperative agent model (Exp. 8), and local confluence (Exp. 9)—all partially validated, identifying specific boundaries (stale marking for confluence, oscillating false finality limited to 1–2 cycles per window for adversarial resilience, Tier 3 non-triggering). The financial consolidation scenario confirms that convergence dynamics are domain-independent. No production-scale validation ($N > 50$ documents) has been performed.

13.4 Final Assessment

As LLM agents become increasingly capable of reasoning, planning, and hypothesis generation, the need for governance-based coordination—rather than pipeline-based orchestration—will grow. Regulated enterprises cannot adopt autonomous agents without the ability to audit, explain, and formally bound their behavior continuously as conditions evolve. The product order $\mathcal{M} = \mathcal{L} \times \mathcal{A}$ provides the formal foundation: governance strictness and convergence progress jointly tracked in a well-founded ordering guaranteeing finite stabilization, while the deterministic reduction kernel ensures every decision is auditable and independent of LLM stochasticity. Validation across two domains—M&A due diligence and financial consolidation with bitemporal restatement—confirms that the convergence mechanism is domain-independent.

Companion work on graph-structured role evidence extends this foundation: it develops layer separation as a response to a monotonicity–fidelity tension in multi-agent estimation (full development elsewhere). Recent networked-LLM opinion experiments [30] independently warn that "consensus" can be far from classical DeGroot optima even when graph spectra still shape transient mixing. A multi-temporal algebraic unification—possibly connecting to concept-lattice views [8] over agent contributions and constraints—remains exploratory. Together with protocol-level adaptability [5], these threads define a research agenda for governed autonomous agent systems at scale in regulated environments.

Acknowledgments

This work was supported by Deal ex Machina SAS. We thank Jean-Christophe Jardinier for critical review of earlier drafts and detailed feedback on the formal convergence arguments. We also thank the communities behind PostgreSQL, NATS, OpenFGA, napi-rs, DSPy, and Mastra for foundational infrastructure.

References

[1] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998. ISBN 978-0521779203.

[2] David Elliott Bell and Leonard J. LaPadula. Secure computer systems: Mathematical foundations. Technical Report MTR-2547, MITRE Corporation, 1973. Foundational lattice-based security model.

[3] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186, 1999. URL `https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance`.

[4] Brian A. Davey and Hilary A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002. ISBN 978-0521784511.

[5] Jose Manuel de la Chica Rodriguez and Juan Manuel Vera Díaz. Self-evolving coordination protocol in multi-agent AI systems: An exploratory systems feasibility study, 2026.

[6] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi: 10.1145/360051.360056.

[7] Marco Dorigo, Eric Bonabeau, and Guy Theraulaz. Ant algorithms and stigmergy. *Future Generation Computer Systems*, 16(8):851–871, 2000. doi: 10.1016/S0167-739X(00)00042-X.

[8] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, 1999. ISBN 978-3540627715. doi: 10.1007/978-3-642-59830-2.

[9] Joseph M. Hellerstein and Peter Alvaro. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9):72–81, 2020. doi: 10.1145/3369736.

[10] Dexter Kozen. Results on the propositional μ-calculus. *Theoretical Computer Science*, 27(3): 333–354, 1983. doi: 10.1016/0304-3975(82)90125-6.

[11] Lindsey Kuper and Ryan R. Newton. LVars: Lattice-based data structures for deterministic parallelism. In *Proceedings of the 2nd ACM SIGPLAN Workshop on Functional High-Performance Computing (FHPC)*, pages 71–84, 2013. doi: 10.1145/2502323.2502326.

[12] Shadaj Laddad, Conor Power, Mae Milano, Alvin Cheung, Natacha Crooks, and Joseph M. Hellerstein. Keep CALM and CRDT on. *Proceedings of the VLDB Endowment*, 16(4):856–863, 2022. doi: 10.14778/3574245.3574268. arXiv preprint `2210.12605`.

[13] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982. doi: 10.1145/357172.357176.

[14] LangChain Team. LangGraph: Building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph`, 2024.

[15] Shulu Li and Edward A. Lee. A preliminary model of coordination-free consistency, 2025.

[16] Aleksandr Mikhailovich Lyapunov. The general problem of the stability of motion. *Annals of the Faculty of Sciences of Toulouse*, 1892. Reprinted in English by International Journal of Control, 1992.

[17] Yihuan Mao, Yipeng Kang, Peilun Li, Ning Zhang, Wei Xu, and Chongjie Zhang. IBGP: Imperfect Byzantine generals problem for zero-shot robustness in communicative multi-agent systems, 2024.

[18] João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents. `https://github.com/joaomdmoura/crewAI`, 2024.

[19] Maxwell Herman Alexander Newman. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics*, 43(2):223–243, 1942. doi: 10.2307/1968867.

[20] OASIS. eXtensible Access Control Markup Language (XACML) version 3.0. Technical report, OASIS Standard, January 2013. URL `https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html`.

[21] Reza Olfati-Saber and Richard M. Murray. Consensus problems in networks of agents with switching topology and time-delays. *IEEE Transactions on Automatic Control*, 49(9):1520–1533, 2004. doi: 10.1109/TAC.2004.834113.

[22] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. Zanzibar: Google's consistent, global authorization system. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC)*, pages 33–46, 2019.

[23] Sergey Pugachev. Codecrdt: Observation-driven coordination for multi-agent LLM code generation, 2025.

[24] Wei Ren, Randal W. Beard, and Ella M. Atkins. A survey of consensus problems in multi-agent coordination. In *Proceedings of the 2005 American Control Conference*, pages 1859–1864. IEEE, 2005. doi: 10.1109/ACC.2005.1470239.

[25] Chaoyi Ruan, Yiliang Wang, Ziji Shi, and Jialin Li. Reaching agreement among reasoning LLM agents, 2025.

[26] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000. ISBN 978-1558604360.

[27] Rudolf Wille. Restructuring lattice theory: An approach based on hierarchies of concepts. In Ivan Rival, editor, *Ordered Sets*, pages 445–470. Reidel, 1982. doi: 10.1007/978-94-009-7798-3_15.

[28] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023.

[29] Jiaxi Yang, Mengqi Zhang, Yiqiao Jin, Hao Chen, Qingsong Wen, Lu Lin, Yi He, Srijan Kumar, Weijie Xu, James Evans, and Jindong Wang. Topological structure learning should be a research priority for LLM-based multi-agent systems, 2025.

[30] Iris Yazici, Mert Kayaalp, Stefan Taga, and Ali H. Sayed. Opinion consensus formation among networked large language models, 2026. cs.SI; also cs.MA, eess.SP. Submitted January 2026.

[31] Lifan Zheng, Jiawei Chen, Qinghong Yin, Jingyuan Zhang, Xinyi Zeng, and Yu Tian. Rethinking the reliability of multi-agent system: A perspective from Byzantine fault tolerance, 2025.

A Implementation Details

```
{
  "timestamp": "2025-08-15T10:32:00Z",
  "agent_id": "facts_extraction_v2",
  "event_type": "claim_emitted",
  "payload": {
    "entity": "NovaTech AG",
    "relation": "annual_recurring_revenue",
    "value": "EUR 50M",
    "confidence": 0.92,
```

```
    "source_doc": "analyst_briefing.pdf"
  }
}
```

```
mode: YOLO    # YOLO | MITL | MASTER
rules:
  - when: { drift_level: [medium, high], drift_type: contradiction }
    action: open_investigation
transition_rules:
  - from: DriftChecked
    to: ContextIngested
    block_when: { drift_level: [critical] }
```

```
{
  "decision_id": "a3f8c...",
  "policy_version": "sha256:7e4f2a...",
  "result": "allow",
  "reason": "no blocking rule",
  "binding": "yaml"
}
```

B Test and Benchmark Counts

341 Rust and 407 TypeScript tests: convergence mathematics and Lyapunov analysis (57 Rust), finality gates and condition evaluation (32 Rust), governance kernel and lattice admissibility (32 Rust), state graph CAS (11 TS), convergence tracker (28 TS), finality evaluator (13 TS), governance agent modes (19 TS), embedding pipeline (11 TS), and supporting infrastructure. Vector finality: 20 Rust + 13 TypeScript tests; witness `vector_finality_blocks_compensation` ($\mu = (1.0, 0.80, 1.0, 1.0)$).

Vector finality validation experiments: (1) exp-ab: scalar vs vector on M&A and financial scenarios; (2) exp8-compensate: compensation attack test; (3) Exp 9 sub-test 7: determinism and replay, Gate G_A^d and $F(t)$, $F^*(t)$ deterministic on same state.

7 convergence benchmarks:

Scenario	Rounds	Outcome
Steady convergence ($\sim$5%/round)	15	Monotonic, ETA ≈ 0
Plateau at 0.70	10	Stagnation detected
Spike-and-drop	–	Gate blocks premature finality
Divergence	–	Negative α, zero progress
One-dimension bottleneck	–	Pressure identifies blocker
Fast convergence ($\geq$0.92 in 3)	3	No false plateau
Empty graph	–	Safe defaults

C Agent Roster and Activation Filters

Facts: Extracts claims, goals, risks (LLM + optional GLiNER2/NLI). Activated by `sequence_delta`. **Drift:** Four drift types, severity classification. `hash_delta`. **Planner:** Ranked proposals, pressure-directed routing. `hash_delta`. **Status:** $V(t)$, α, plateau detection, HITL routing. `timer`. **Governance:** Three-tier routing (Definition 8, Section 4.4), circuit breaker 3 failures/60s. **Executor:** CAS transitions, WAL, monotonic upserts. **Tuner:** Activation filter optimization. Filter types: `sequence_delta`, `hash_delta`, `timer`, `pressure_directed`. Agents emit events; no direct calls.

D Project Horizon: Detailed Scenario and Results

Five documents: (1) Analyst Briefing (baseline); (2) Financial DD (ARR 50M→38M, governance block); (3) Technical (CTO departure, medium drift); (4) Market (patent suit); (5) Legal (partial resolution, HITL). Key metrics: 5 documents, 312 facts, 18 contradictions (15 agent-resolved, 3 human), 12 rounds, 240s, 487 audit events, 156 policy evaluations, 3 governance blocks.

E Full Experimental Result Tables

Exp. 1: Epochs 1–4 (Score 0.75, V 0.25); 5 (0.808, 0.148); 9 (1.0, 0.0); 10–12 (0.927, 0.022); 13 (1.0, 0.0); 14–16 (0.956, 0.008); 17–19 (1.0, 0.0); 20–28 (0.956, 0.008). Exp. 3: Gates enabled 19 RESOLVED, 4 ESCALATED, 20 HITL, 14 ACTIVE; gates disabled 23 RESOLVED, 0 ESCALATED, 37 HITL. Exp. 2: $N = 10, \rho = 0.1$: 21 conv pts, 20 decisions; $N = 100, \rho = 0.3$: 45 conv pts, 34 decisions, 32 epochs; $N = 500, \rho = 0.1$: V 0.251→0.001 (1 resolution batch, 50 edges, ∼2 min); $N = 500, \rho = 0.3$: V 0.251→0.001 (2 batches, 150 edges, ∼2 min, 63 propagation epochs). Exp. 7: YOLO 31/32 Tier 1; MITL 49/49 Tier 2; MASTER 28/40 Tier 1, 12 rejections. Exp. 8: Baseline 0 false RES; Inflate 10; Collude 18. Exp. 9: Confidence ratchet, idempotency, kernel determinism Pass; CRDT partial (6/24 confluent); eventual consistency Pass. Financial: V 0.25→0.00 at epochs 24, 28–30, 36; 45 conv pts, 38 epochs.

F Formal Assumptions, Limitations, and Hardening

Resolved (Paper 1 scope). Proposition 1 intra-epoch conditioning is resolved: the paper uses the rank function $r(t)$ and explicit two-regime framing (intra-epoch vs. cross-epoch); $r(t)$ is non-increasing within an evidence epoch when at least one dimension makes progress. The proof-sketch gap (dimension 4 / new context injection) is resolved in this framing. Vector finality $F^*(t)$ (Section 5.4) is implemented; PO-2 (non-compensability) is proven by the `vector_finality_blocks_compensation` witness. The connection between $V(t)$ and finality is stated in the main text: $V(t) \to 0$ is necessary but not sufficient for finality; $F^*(t) = \text{true}$ is necessary but not sufficient for $V(t) = 0$. The progress indicator α is renamed from "convergence rate" and flagged as unreliable in non-stationary regimes. MASTER mode semantics (most restrictive, not a bypass) are consistent with the implementation.

Remaining limitations (Paper 1). **Trajectory quality Q:** The constants (e.g. 0.12, 5, 0.65, 0.85, -0.3) are domain-configurable (finality.yaml) and have no formal justification; Experiment 3 shows the system remains in ESCALATED/HITL under spike-and-drop and oscillating patterns for $\pm 20\%$ variation in Q. **Dimension weights:** Weights (0.30, 0.30, 0.25, 0.15) apply only to the scalar goal score used for the near-finality band (HITL trigger); the formal finality predicate uses per-dimension thresholds τ_d, not weights—the formal result does not depend on weight selection. **Complexity:** The comparison with BFT is reframed as eliminating the n^2 factor per round; total complexity is $O(nk)$ with k input-dependent (e.g. $k = 12$ in validation). **Machine-checked proofs:** Proposition 1 and Theorem 1 remain proof sketches; no proof assistant has checked them. **Gate naming:** The paper (Gates A–E: monotonicity, evidence, trajectory, quiescence, minimum content) is canonical; some supporting design docs use different labels and should be aligned to the paper.

Implementation and scalability. Dual temporality: schema complete; valid-time population from unstructured text may require temporal NLP. Evidence schemas and obligation handlers are framework-ready, per-deployment. Single Postgres/NATS/process validated; horizontal scaling and graph scale (e.g. $N > 500$ documents) not validated.

Assumptions A1–A7. A1 discretization (empirically validated); A2 kernel determinism (Rust test suite); A3 CRDT monotonicity (SQL guards and invariants); A4 progress precondition (Exp. 6); A5 local confluence (Exp. 9 partial—stale marking non-commutative, eventual consistency after re-extraction); A6 non-compensability **closed** by vector finality $F^*(t)$ and PO-2; A7 tier completeness (Exp. 7 partial—Tier 3 not triggered). Extended experiment program: E1–E5 planned (scalar vs vector, Tier-3 reachability, ϵ sweep, confluence, adversarial).

G Enterprise and Regulatory Fitness

Regulated environments—financial services, healthcare, defense, critical infrastructure—impose requirements that go beyond functional correctness. This appendix maps the architecture's capabilities to specific regulatory demands, showing that the mechanisms described above are not merely useful but structurally necessary for enterprise deployment.

G.1 Temporal Audit for Regulators

Regulatory frameworks frequently require point-in-time reconstruction: demonstrating what was known, when, and what decisions followed. SOX Section 404 requires that internal controls over financial reporting be documented as they existed at the reporting date. IFRS 9 (Expected Credit Loss) requires temporal evidence chains for impairment assessments. Basel III mandates historical risk factor traceability.

The bitemporal semantic graph (Section 4.2) enables these queries directly:

- **"What did the system know on audit date T'?"** As-of transaction-time query: filters to rows with `recorded_at` $\leq T'$ and (`superseded_at` $> T'$ or not superseded).
- **"What was believed true for reporting period T?"** As-of valid-time query: filters to rows with `valid_from` $\leq T$ and (`valid_to` $> T$ or open-ended).
- **"What did the system know at T' about facts valid at T?"** Combined bitemporal query: both filters applied simultaneously.

Unlike retrospective log analysis (grepping timestamps in application logs), these are first-class database queries with indexed support, returning structured graph state rather than unstructured log lines.

G.2 Cryptographic Non-Repudiation

When a scope reaches finality (Section 5.8), an Ed25519-signed JWS certificate is issued containing: the scope identifier, the finality decision, the timestamp, and content hashes of the governance and finality policy files in effect. This certificate is self-contained: an external party—auditor, counterparty, regulator—can verify the signature and inspect the payload without access to the running system.

The policy-version hashes are critical: they bind the finality decision to the exact rule set that produced it. If governance rules are amended between two finality decisions, the certificates will carry different policy hashes, enabling precise attribution of which decisions were made under which rules.

This addresses requirements in MiFID II (best execution record-keeping), SOX (management sign-off chains), and EMIR (trade repository reporting), where demonstrating *which rules governed a decision* is as important as demonstrating the decision itself.

G.3 Summary of Remaining Regulatory Mappings

Several additional enterprise concerns—policy change management (each `DecisionRecord` binds to a content-hashed `policy_version`), ongoing compliance lifecycle (the perpetual finality mechanism produces certificate chains for KYC/AML, IFRS 9, Basel III, and pharmacovigilance monitoring), evidence freshness (Gate B enforces per-domain staleness limits), separation of duties (OpenFGA + governance modes map to SOX/PCI-DSS segregation requirements), operational resilience (circuit breaker, message deduplication, graceful degradation), and governance agility (YAML rule changes are auditable and require no recompilation)—are addressed by mechanisms described in Sections 4.3–5.9 and are not re-derived here. These mappings are assertion-based: they follow from the architectural properties established above but have not been validated in production regulatory contexts.